

AQA A Level Computer Science NEA

Shayan Nezami

March 25, 2026

Contents

1	Analysis	5
1.1	Introduction	5
1.2	Client Overview	5
1.3	Client Interview	5
1.4	Specification	7
1.5	Technical Points	9
1.6	Program Structure Diagram	9
1.7	Transmission Procedure	11
1.7.1	Key Exchange	11
1.7.2	Symmetric Encryption	14
1.7.3	Verifying the Integrity of Communications	14
1.8	Compression	14
1.9	Class Diagram	17
1.10	Order of Completion	17
2	Documented Design	19
2.1	Huffman Compression	19
2.1.1	Encoding and Decoding	19
2.1.2	Reading and Writing Bytes	29
2.1.3	UML Class Diagram	35

2.2	RSA Key Exchange	37
2.3	Verifying Communication Integrity	43
2.4	Transmission Procedures	44
2.4.1	UML Class Diagram	51
2.5	Full UML Class Diagram	51
3	Technical Solution	53
3.1	Main Program	53
3.2	Huffman Compression	61
3.3	Transmission Procedure	69
4	Testing	77
4.1	Testing Strategy	77
4.2	Typical Use Video	77
4.3	Verifying the RSA Key Exchange	82
4.3.1	The Miller-Rabin Primality Test	82
4.3.2	The Extended Euclidean Algorithm	85
5	Evaluation	89
5.1	Evaluation Against the Requirements	89
5.2	Client Feedback	92
5.2.1	Final Client Interview	92
5.2.2	Evaluating the Interview	93
5.3	Improving the Outcome	94

List of Figures

1	Program Structure Diagram	10
2	Proposed UML Class Diagram	17
3	A flowchart depicting my Huffman encoding procedure	20
4	A flowchart depicting my SearchTree procedure	21

5	A flowchart depicting the creation of the Huffman tree	23
6	A flowchart depicting my Huffman decoding procedure	25
7	A flowchart depicting the reconstruction of the Huffman tree	26
8	A UML class diagram for the compression program	36
9	A flowchart for the Miller-Rabin algorithm	38
10	A flowchart for the procedure to generate an RSA prime	39
11	A flowchart for the Extended Euclidean Algorithm	41
12	A flowchart for the procedure to generate RSA keys	42
13	A UML class diagram for the Transferer, FileSender, and FileReceiver classes . .	51
14	A UML class diagram for the whole program	52

Listings

1	Code for the SearchTree procedure	21
2	Code for creating the Huffman tree	23
3	Code for reconstructing the Huffman tree	27
4	Pseudocode for the HuffmanNode class	27
5	Code for the HuffmanNode class	28
6	Pseudocode for the ByteMaker class	29
7	Pseudocode for the ByteReader class	31
8	Code for the ByteMaker class	32
9	Code for the ByteReader class	34
10	Code for the Miller-Rabin algorithm	38
11	Code for the procedure to generate an RSA prime	39
12	Code for the Extended Euclidean Algorithm	41
13	Code for the procedure to generate RSA keys	42
14	Pseudocode for the hashing procedures	43
15	Code for the hashing procedures	43
16	Pseudocode for the serialisation procedures	44
17	Code for the serialisation procedures	45

18	Pseudocode for the transfer classes	46
19	Code for the transfer classes	48
20	Program.cs	53
21	Client.cs	54
22	Server.cs	59
23	Huffman/ByteMaker.cs	61
24	Huffman/ByteReader.cs	62
25	Huffman/HuffmanEncoder.cs	64
26	Huffman/HuffmanDecoder.cs	66
27	Huffman/HuffmanNode.cs	68
28	Transmission/Transmission.cs	69
29	Transmission/Transferer.cs	70
30	Transmission/RSA.cs	72
31	Transmission/SetupAes.cs	75
32	Transmission/Hashing.cs	76
33	Code for testing prime number generation	83
34	Prime number generation code	83
35	Prime number generation test output	85
36	Code for testing my Extended Euclidean Algorithm	86
37	Extended Euclidean Algorithm code being tested	86
38	Extended Euclidean Algorithm test output	87

1 Analysis

1.1 Introduction

A backup utility is an essential software program for any computer system, allowing users to keep their files and data safe in the case of hardware failure, or various possible software issues. It is particularly important that these files are stored externally, on another device, in case of such hardware failure. Therefore, I will create a backup utility for storing files on an external server from a client PC. This will require various components, such as encrypting the files stored on the server, encrypting the transmission of files between client and server, and compressing the files being transferred.

The main benefits of the program I will create over existing systems are that my utility will be fully open source, so the function of the program can be independently inspected by any user, and that it will be tailored to my client's exact needs: many existing applications are either closed source, or too bloated with unnecessary features and unintuitive interfaces. Instead, this will be a simple, easy-to-use command line tool.

I will research this through two main methods: a discussion with my client, and using reliable external sources. The latter may come from reputable books, websites, papers, or journals, and will be cited.

1.2 Client Overview

My client will be a classmate – Sam Yarasani – who informed me he is looking for such software. Therefore, I will tailor this program for his exact needs, and I will establish his specific requirements through a client interview. This has the benefit over existing systems of being in exactly the format he would like, without additional features which add complexity and bloat to the final program. Plus, it will have the exact user interface he would like.

1.3 Client Interview

Below is a record of the transcript of my first interview with my client. I will use this to inform the design requirements for this backup utility.

Q: I understand that you are looking for a backup utility?

A: Yes, I'd like to be able to create external backups of my system on a separate device.

Q: What devices would you like to create backups of?

A: My laptop and desktop PC.

Q: What operating system are those devices running?

A: Arch Linux, and Ubuntu 22.04 respectively, but I would like this to work on any modern Linux distribution - I may want to change in the future!

Q: Where would you like the backups to be stored?

A: I have a fairly old laptop which I use to run a few home server utilities. I'd like to also store the backups here, if possible.

Q: Can you be more specific about the laptop?

A: It's around 15-20 years old, I'm not exactly sure, with 4GB of RAM. Also, it's running Ubuntu server.

Q: Would you like a third-party server in a data centre to store the backups instead?

A: Absolutely not. The key here is that this is all fully local to me, and that I am the only one with access to the backups. Many current backup utilities lack this!

Q: Would you want the files stored to be encrypted in transmission, and while stored?

A: It is crucial for the files to be hidden while being transferred, as I do not want any data being intercepted by others. However, I don't need the files to be encrypted while being stored on the server, as I own that server and know that it is kept safe.

Q: Do you have any other priorities?

A: File transfers should be as quick as possible, and as little space as possible should be used on the server device. Oh, and I want to be sure that transfers have been successful.

Q: What type of files will you be backing up?

A: Well, I will be using this for my whole system, so all sorts of files. I guess there will be text files, like system configuration files, image files, video files, and raw binary executables.

Q: Okay. The non-text files you were talking about, are they already compressed?

A: Yes actually, almost all of my audio and video files are already compressed.

Q: Very good, thanks. What about the user interface, what form should this take?

A: I want a very simple command line interface, with no messy graphical elements. It could have an interactive mode, but there should certainly be a way to use it non-interactively, with just command line flags, so that I can use it in scripts.

Q: Finally, is it important to you for this project to be finished by a certain date?

A: Absolutely. It is essential for this to be finished by the end of March 2026. This is one of my key priorities, since I will start work on an important project in April, and I need to ensure I have safe backups of my files by its start.

I have summarised my key findings from this interview below:

- The program should work on Linux systems,
- The program should be able to comfortably run on my client's devices, without putting much strain on them,
- I must also code the server storing the files, which should run on a device my client also owns,
- The files must be encrypted in transmission,
- The files should be compressed before transmission, and decompressed upon retrieval by the client, so that the transmission is as quick as possible, and as little space as possible is taken up on the server,
- Only text files need to be compressed, as most other files are already compressed,
- There should be no graphical user interface, but instead a command line interface that can be used non-interactively,
- There is a hard deadline of the 31st March 2026 for finishing the project.

1.4 Specification

Based on the requirements established in the client interview, I have established the following specification points. These will guide me throughout the development process, and can serve as a metric against which the final product is evaluated.

1. The program should allow the user to create file backups on a server, run by the client themselves,
 - (a) The software should be able to run on both the client and server, with appropriate functionality for each situation,
 - (b) They should be able to work together, with a server being able to store the user's backed up files,
2. The client program should run on modern Linux systems, usable on a laptop or desktop computer,
 - (a) An installable binary package should be created that is executable on amd64 Linux systems,
 - (b) It should be very unlikely for the client to use more than 50MB of RAM for prolonged periods of time,

- (c) It should be very unlikely for the client to use more than 5% of CPU time for prolonged periods of time while running on a modern laptop; my client's processor, the Intel i9-12950HX, can be used for a test case,
- 3. It should be usable with an intuitive command line interface, not requiring interactive input,
 - (a) All required functions of the program should be able to be passed in through command line flags and parameters,
 - (b) There can be an optional interactive mode, but this should not allow any functionality than the non-interactive mode does not,
 - (c) There should be no graphical interface,
- 4. The server should be able to run on a low-end laptop 10-15 years old,
 - (a) The program should avoid reading and writing to the system drives as much as possible, since most such devices use HDDs,
 - (b) It should be very unlikely for the server program to use more than 200MB of RAM for prolonged periods of time,
 - (c) It should be very unlikely for the server program to use more than 10% of CPU time for prolonged periods of time while running on such a device; my client's desired server's processor, a 2nd generation Intel i5 processor, can be used as a test case,
- 5. All file transfers should be secure, so a man-in-the-middle would not be able to intercept data,
 - (a) Any data intercepted should be of a format that the interceptor would not be able to decipher, or gain any useful information from,
 - (b) The methods used should have their correctness checked,
- 6. Text files stored on the server should be compressed such that they use a reasonable amount of storage,
 - (a) They should be at most as large as (or very close to) the uncompressed files, i.e. the file size should never increase following compression by more than 10 bytes,
 - (b) The client should be able to decompress any file when needed by the user, storing any needed details about the compression within the file,
- 7. File transfers should be verifiably correct, so data is not corrupted during transmission,
 - (a) Once a file has been transferred to the server, it should be compared with the original file on the client device,
 - (b) This checking should be with a much smaller version of the file than that which was originally sent for most files – it should be up to 10kB in size,
- 8. Generally, the program should work robustly,
 - (a) Problems with user input or command line arguments should be gracefully handled,

- (b) The program should not crash while running,
- (c) Errors should be reported to the client, server, or both, depending on which is appropriate in the situation.

1.5 Technical Points

Based on the specification points established, I have extracted the key technical challenges that will be involved in building the software below.

1. Implement a transmission system with asymmetric encryption for key exchange between the client and the server,
2. Implement symmetric encryption between the client and the server after keys have been established,
3. Use a method to verify the integrity of the files transferred,
4. Implement a lossless compression algorithm for the text files stored on the server.

1.6 Program Structure Diagram

This diagram (Figure 1) shows how these main technical blocks will interact with each other in the client and server programs. While they will run different main procedures, these will share some key components.

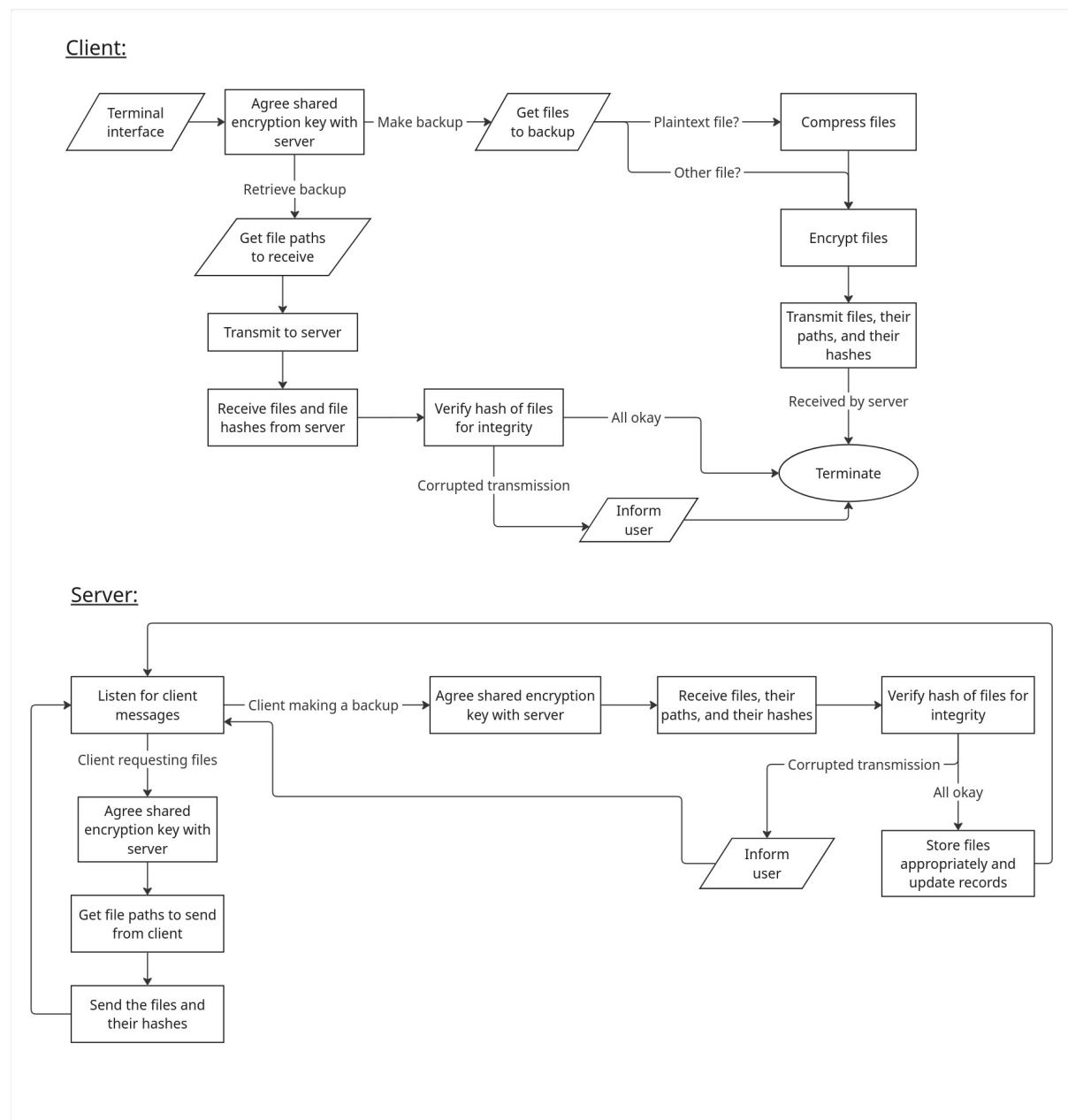


Figure 1: Program Structure Diagram

I will now consider how I can implement these main blocks.

1.7 Transmission Procedure

This is one of the largest components of the program, which is used for any and all communications between the client and server devices. There are some key points that need to be individually considered:

1. There needs to be a procedure used to exchange keys between the sender and receiver, perhaps in a similar style to the RSA algorithm, providing asymmetric encryption,
2. There must be a symmetric encryption algorithm used to encrypt communications following the establishment of keys,
3. The communications should be verified to be correct, using a method to check that there has not been any corruption during data transfer.

1.7.1 Key Exchange

The three algorithms I will consider for this are RSA, Diffie-Hellman Key Exchange, and Elliptic Curve Cryptography.

These algorithms are commonly used, and very well documented. All three rely on mathematical problems that are intractable to solve without some prior knowledge: RSA relies on the difficulty in factoring a large semiprime integer (one with exactly two prime factors), while Diffie-Hellman and elliptic curve systems rely on the discrete logarithm problem.

While elliptic curve cryptography can provide equivalent security with much smaller keys, and requires less processing power for the sender and receiver, this is unlikely to be at all impactful on the small scale of this backup utility, where this asymmetric encryption is just used to share a symmetric encryption key. However, these algorithms are tough to implement, and involve very high-level mathematics. This could increase the time taken to deliver the program past the client's deadline, and add more avenues for potential bugs to develop. This can be very dangerous, as shown in the following paper [1], where a small error in the implementation led to researchers being able to extract the full key.

Plus, implementing it could cause me to fall short of my deadline, and it is very important for my client that I do not. Therefore, I will exclude elliptic curve cryptography from my choices.

I will now consider the basic implementation details of the RSA and Diffie-Hellman algorithms to aid my decision-making.

RSA

The website [2] contains details about the mathematics of the RSA algorithm, and how it can be implemented. From this, I have generated the following steps that are taken in the RSA algorithm.

1. The receiver generates two large primes, p, q , then computes their product, $n = pq$.
2. Euler's Totient function, $\phi(pq) = (p-1)(q-1)$, is applied to n .
3. An encryption exponent, e , is selected such that $1 < e < \phi(n)$, and e is relatively prime with $\phi(n)$. Note that $a, b \in \mathbb{Z}^+$ are relatively prime $\iff \nexists x \in \mathbb{Z} : x > 1, x|a, \text{ and } x|b$.
4. A decryption exponent, d , is calculated such that $de = 1 \pmod{\phi(n)}$. The extended Euclidean algorithm provides a method for this.
5. The public key is formed from (n, e) and the private key (n, d) . The receiver sends their public key only to the sender.
6. The encrypted ciphertext, C , is calculated through $C = M^e \pmod{n}$, where M is some numerical representation of the message.
7. The ciphertext is decrypted into the plaintext message, M , by the calculation $M = C^d \pmod{n}$.

Now, the sender encrypts their message with the receiver's public key, and then only the receiver can decrypt it since only they have the private key. Computing $\phi(n)$ is infeasible without knowing the primes, p, q , that n is a product of, and this is also infeasible to compute due to the large size of the primes. This means no other party can calculate the private key from the public key.

As mentioned in the steps, the Extended Euclidean Algorithm is key to RSA, generating the decryption constant d . Wikipedia [3] provides an explanation of the algorithm. In effect, it calculates coefficients x, y , such that $ax + by = \gcd(a, b)$ for some a, b .

This is useful, since the decryption exponent, d , is calculated such that $de = 1 \pmod{\phi(n)}$. Therefore, by choosing $a = e$ and $b = \phi(n)$, the algorithm calculates $ex + \phi(n)y = \gcd(e, \phi(n)) = 1$ since in RSA e (or n) is chosen such that $\gcd(e, \phi(n)) = 1$. Now, taking both sides in the modulo of $\phi(n)$, we get $ex = \gcd(e, \phi(n)) \pmod{\phi(n)}$, so x , that the algorithm calculates, is the desired decryption constant d .

Diffie-Hellman key exchange

Page 84 of the following lecture notes [4] contains a brief description of the mathematics behind the Diffie-Hellman key exchange. Below I describe the key steps in this algorithm in my own words.

1. Either the sender or the receiver randomly generates a large prime, p , and a large number, g , and sends these to the other.
2. The sender and receiver then generate random secrets s_a and s_b respectively, for $s_a, s_b \in \mathbb{Z}^+$.
3. The sender sends $a = g^{s_a} \pmod{p}$ to the receiver, who computes $a^{s_b} = g^{s_a s_b} \pmod{p}$.
4. The receiver sends $b = g^{s_b} \pmod{p}$ to the sender, who computes $b^{s_a} = g^{s_a s_b} \pmod{p}$.

5. Now, they both have the shared secret, $S = g^{s_a s_b} \pmod{p}$, which can be used to establish a shared key.

An attacker would have access to g , p , $g^{s_a} \pmod{p}$, $g^{s_b} \pmod{p}$, and need to compute $S = g^{s_a s_b} \pmod{p}$ to compromise the conversation. A quick solution to the Discrete Logarithm Problem would imply a quick solution to this problem, but one is not currently known. This is central to the security of this key exchange, similarly to the prime factorisation problem for RSA.

As can be seen, RSA and Diffie-Hellman are very similar in security, and difficulty of implementation, however the advantage of RSA is that it can also be used to provide signed certificates to prove the authenticity of the sender. This has not been an aim as of yet, but using RSA will make it easier to implement this if desired in the long-term, since I intend to make this project open-source upon completion. Therefore, I will use the RSA algorithm for key exchange.

An important aspect of the RSA (and Diffie-Hellman) algorithms is the generation of suitably large, random, prime numbers. It is completely infeasible to go through all of the possible factors of a prime to generate primes of the size required (512 bits), so more complicated algorithms need to be used for this task. Generally, a large random integer is created by filling an array of bytes with random bits, then it is tested for primality, and this is repeated until a prime is found. The algorithms to be mentioned are used to check whether the integers are prime.

One option is the AKS primality test. Wikipedia [5] provides a description of it. The algorithm is based upon polynomial ring theory, and can deterministically verify whether a given integer is prime in polynomial ($\mathcal{O}(\log(n)^{12})$) time. However, despite this, the runtime is still normally very long, and so it is not typically used in practice - it is mainly analysed in research contexts instead. Plus, it depends on knowledge of very advanced mathematics that I do not have, and learning them to implement this may cause me to exceed my client's strict deadline. Therefore, I will not be using this algorithm.

Another possible algorithm is the Fermat primality test, which is based on Fermat's Little Theorem. This states that $a^{p-1} = 1 \pmod{p}$ for any integer a and primes p . By repeating this test for many integers a , a number is very likely to be prime if it satisfies the equation every time. However, a key weakness of this algorithm is that it can falsely identify certain numbers, such as the Carmichael numbers, as prime.

Therefore, the best algorithm to use in my situation is the Miller-Rabin primality test. It uses a similar, though slightly more complicated, equation, and checks whether an integer satisfies the equation for a large number of bases, in which case it flags it as probably prime. The advantage it has over the Fermat test is that there are no known sets of composite numbers that are falsely identified as prime for all bases a [6]. In each test, there is only a $\frac{1}{4}$ chance that a composite number is falsely flagged as prime, so by repeating the test 40 times, the probability of an incorrect result is reduced to 4^{-40} , which is astronomically small. Therefore, I will use the Miller-Rabin primality test to generate large random primes for the RSA algorithm.

1.7.2 Symmetric Encryption

Following the establishment of keys, it is greatly advantageous to use a symmetric encryption system rather than continuing with asymmetric systems, since these are much faster and more efficient. A similar process is used by the TLS protocol, which secures more internet communications.

There are a great number of available symmetric encryption algorithms, including the Caesar Cipher, AES, ChaCha20, and Blowfish, among others.

The Caesar cipher can be immediately eliminated, since it is trivial to break with even very simple cryptanalysis. However, the other algorithms are very similar in that they all provide secure encryption that is very tough to break. The final choice of algorithm should be one that is well-documented, simple to implement, and efficient to run.

However, AES has been used as the basis of internet security for decades, and as such there has been great research into verifying its security, and ensuring implementations do not contain vulnerabilities. Plus, C# provides a standard implementation of this. Therefore, I will use AES for my symmetric encryption needs.

1.7.3 Verifying the Integrity of Communications

It is necessary to ensure all communications have arrived at the desired recipient uncorrupted, exactly as they were sent. Therefore, some method must be used to ensure that this is the case.

Some simple methods for this include majority voting, where each bit is sent an odd number (> 1) of times, and the most common bit is accepted, and parity bits, where a bit is added to the end of each string of defined length to ensure that the number of 0s or 1s is either even or odd as desired.

However, these methods allow multiple mistakes to “cancel” each other out, and say little about the correctness of the whole message. A more robust method is to compute some form of hash from the message, and send that directly after it. The receiver will also compute a hash of the data received, and compare it to the hash sent. If these match, then the data has arrived intact. This verifies that the whole message has arrived safely, while requiring comparatively little additional data to be transferred. Plus, these algorithms can run very quickly.

I can use the SHA-256 algorithm for this. It is a cryptographically secure hash function, that produces a 256-bit hash for any length of text. It is widely used, and it is very unlikely for two distinct inputs to give the same output. There is also an available C# implementation that I can use for this.

1.8 Compression

Firstly, it is important to establish that I will want to use a lossless algorithm, since a backup program should not change the files being backed up – the whole point is to provide a safe copy

of the actual system files themselves. Plus, lossy compression of important system files would be catastrophic.

There is also the key problem that many files being backed up will already be compressed. The system must be able to deal with these, not producing output with larger file size than the uncompressed versions.

There is also the important consideration of whether each file is individually compressed, or whether the whole block of files is compressed together.

There are two broad options I can take based on the above. I could either compress the whole block of data, or each file individually, with some metadata at the beginning of the file outlining what procedure (if any) was used.

The first option will not be very useful, since different file types can be effectively compressed using vastly different algorithms, so I will not be able to achieve significant size reductions in this way. Therefore, I believe the right choice is to compress files individually.

This means that I would need to write different algorithms for different file types, and have a way of detecting files that have already been compressed. However, the discussion with my client revealed that he already compresses the majority of non-text files on his system. So, I have decided to just implement compression for plaintext files, where the most space savings can be made, while minimising the unnecessary processing taking place, so the program executes as quickly as possible. This will also reduce development time, allowing me to finish the program in time for my client's strict deadline.

Three lossless plaintext compression algorithms I will consider as the basis of my algorithm are Run-Length Encoding, Huffman Coding, and Prediction by Partial Matching.

Run-Length Encoding works by representing repeated runs of one character as frequency-data pairs. For example, AAAAAABBBB would be stored as "6A3B". However, this is not very effective for text files, where there is not often repeated runs of characters; it is instead very useful for images, where there may be, for example, a dark background with many long runs of dark pixels. Therefore, it is unsuitable for my needs, to compress plaintext.

Prediction by Partial Matching uses complex statistical techniques to predict the most probable next characters based on the previous ones, assigning more probable characters shorter binary representations. However, to create and optimise such an algorithm, it would take a long time, which may cause be to pass my client's non-negotiable deadline. Plus, it is more computationally complex than the alternatives, so the compression and decompression algorithms would use more memory and processor time, and would likely take longer to run.

Huffman Coding provides a perfect balance between complexity and how effective it is at compressing plaintext. It looks at the file as a whole, assigning more frequently used characters shorter binary representations. This is very effective for text files, where some letters are used more often than others. Moreover, most available Unicode characters are never used (special characters, scientific symbols, foreign language characters...) in most files, so the majority of characters would have a shorter representation in a file compressed in this way. While it may be more complex than Run-Length Encoding, it is perfectly feasible for me to implement it in

the given time, and the implementations can be very quick. Even though Prediction by Partial Matching can in some cases provide better compression, for the files where this really matters to my client, they can just compress them themselves before backing them up.

I have researched the Huffman Coding algorithm through this website [7], which gives an explanation of one implementation of it.

The high-level steps of the encoding algorithm are as follows:

1. Count the number of times each character appears in the text,
2. Turn each character into a tree with a single node, weighted by its frequency,
3. Continually merge the two nodes with the smallest weights into a new tree, creating a new node with the nodes being merged as children, with its weighting being the sum of its two children's weights,
4. Stop once there is only one tree left - this is the Huffman tree,
5. Carry out a post-order tree traversal on the Huffman tree to write a representation of it to the beginning of the compressed file, and to build up a mapping of characters to their binary Huffman representation (this is defined by their placement in the tree with respect to the root node),
6. Go through each character in the file, writing its Huffman representation to the compressed file.

The high level steps of the decoding algorithm are as follows:

1. Read the Huffman tree from the start of the compressed file, representing it as a tree in the program,
2. Carry out a post-order traversal of the tree to build up a mapping of the Huffman binary bit-strings to the characters they represent,
3. Go through every bit in the compressed file, writing the character they represent to the decompressed file.

The final consideration that is needed is whether the compression will occur on the client device, before being sent to the server, or on the server, having been received. Compression on the client end is the better choice for two key reasons. Firstly, it would mean that smaller files are transferred, resulting in quicker transmissions and minimising the chances of errors occurring during transmission. Secondly, my client is planning to use a much more powerful device as their client than their server, so it would be beneficial to carry out as much computation as possible there, rather than on the much older server.

1.9 Class Diagram

Figure 2 shows a proposed design for the classes in my program.

The FileSender and FileReceiver classes will contain methods related to sending and receiving data. These methods can be used by both the client and server programs, and so the Client and Server classes - which will contain the main methods run by the client and the server respectively - depend on them. These classes both inherit from an abstract base FileTransferer class which will contain any methods or attributes common to both classes.

The HuffmanNode is a custom data structure representing the nodes that form a Huffman tree. This is used by both the HuffmanEncoder and HuffmanDecoder classes, which contain methods for encoding plaintext into Huffman, and decoding Huffman compressed text back into plaintext respectively. The ByteMaker and ByteReader classes will deal with any complexities that may arise in the process of reading from and writing to binary files.

Finally, the SetupRSA class will contain methods and data relating to the creation of RSA public and private keys, SetupAES will establish the shared AES encryption key, the Hashing class will allow methods for generating and verifying hashes, and the Transmission class will contain common procedures needed to serialise data into a format that can be decoded once received, and to decode such data once it has been received. These are of use in the FileSender and FileReceiver classes.

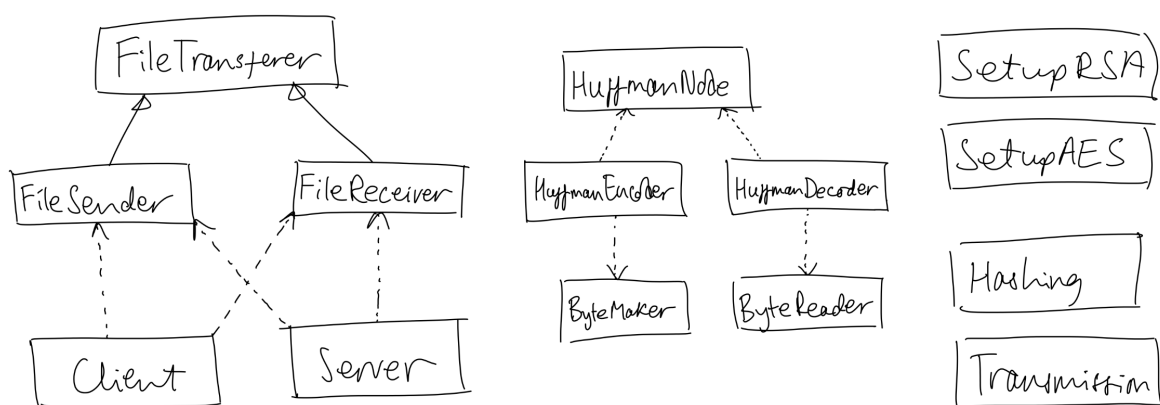


Figure 2: Proposed UML Class Diagram

1.10 Order of Completion

This is a rough timeline of how long I expect it would take me to complete the sections outlined above.

1. Plaintext compression - 2 weeks
2. RSA style key exchange - 2 weeks
3. Hashing to verify transmission, and symmetric encryption to secure it - 1 week

4. Finalising the transmission system - 2 weeks
5. Terminal interface - 1 week
6. Putting it all together - 2 weeks

In total, this will take 10 weeks, leaving me on track to meet my deadline with plenty of time to spare in case unexpected issues arise.

2 Documented Design

In the design of my program, I expect that the key (though not only) points of complexity and demonstrations of technical skill will arise from the following:

1. The use of a custom class to represent the Huffman tree (a binary tree),
2. Using dictionaries, priority queues, stacks, and recursive subroutines for depth first tree traversal to create and reconstruct the Huffman trees,
3. The reading and writing of bytes to binary files, with the use of logical operations to read or modify bits, and placing metadata at the start of the file to indicate whether a file is compressed, and how many 0 bits have been added at the end of the file as padding (to be ignored upon decompression),
4. The mathematical Miller-Rabin algorithm to generate large random prime numbers,
5. The mathematical Extended Euclidean algorithm to generate the decryption constant for the RSA algorithm.

2.1 Huffman Compression

As decided upon in my Analysis (Section 1.8), I will use Huffman Coding to compress all plaintext files that are being backed up. The compression and decompression will occur on the client-side, before the files are sent, and after they are received.

The benefits of this are twofold. Firstly, this means less data is transmitted, reducing the risk of corruption, and speeding up the backup process. Secondly, as identified in my Client Interview (Section 1.3), my client will run the server on hardware that is significantly less powerful than their laptop, which acts as the client. Therefore, it is advantageous to run as much processing as possible on the client-side.

2.1.1 Encoding and Decoding

This website [7] provides a thorough written explanation of the Huffman algorithm. Using this to better understand the process, I have created a flowchart depicting how I can implement the encoding (Figure 3), and the decoding (Figure 6).

These flowcharts include implementation details, such as how dictionaries, stacks, priority queues, and recursive subroutines are used to carry out some of the high-level tasks described in the source. My implementation also ensures the compression does not (significantly) increase the size of the files, by using a 1-bit flag at the start of the file to indicate whether compression has been used, and not using any compression if it would increase the file size overall. Do note that the final file will still be one byte larger in this case due to the extra bit added, but this is insignificant and unavoidable.

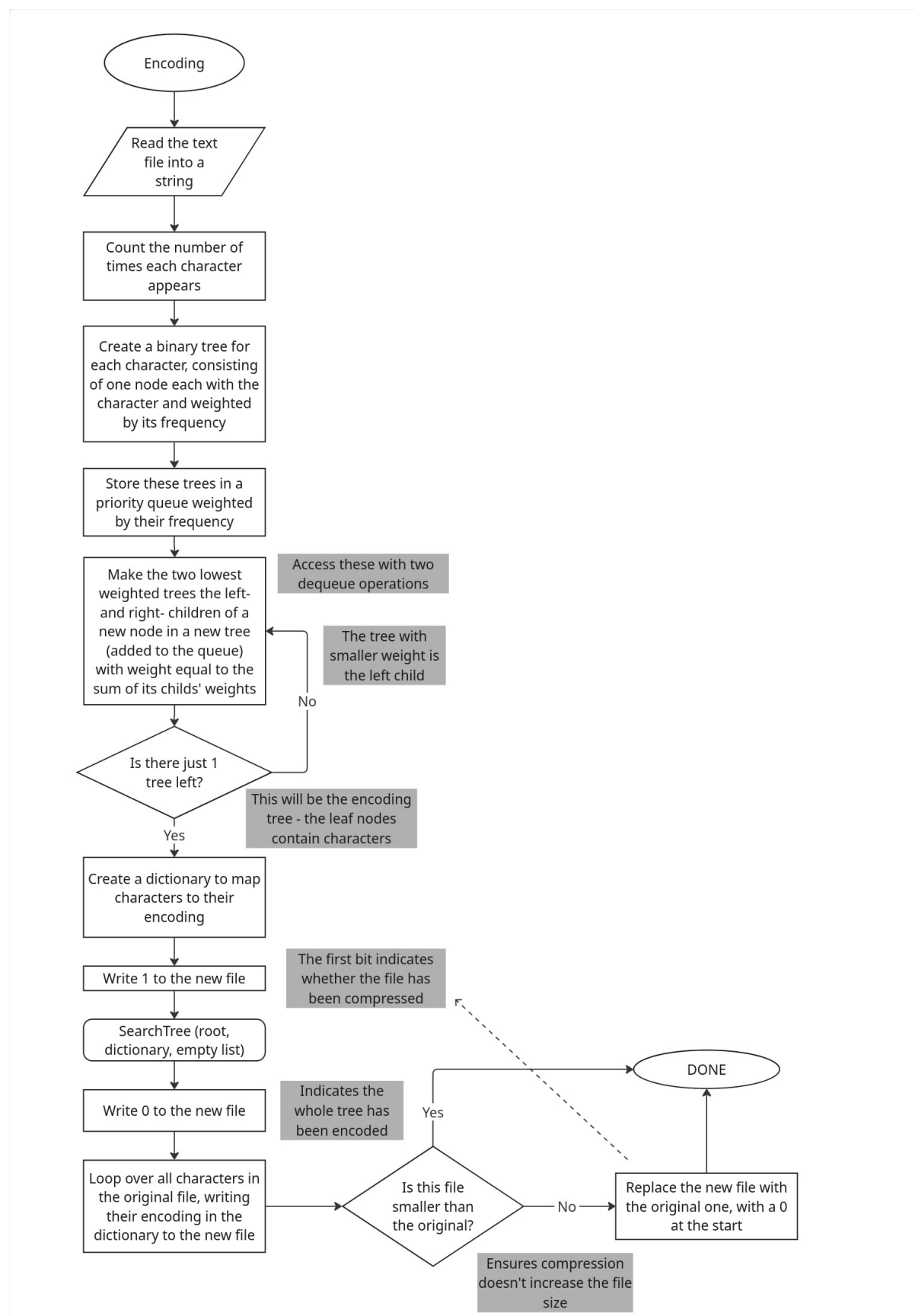


Figure 3: A flowchart depicting my Huffman encoding procedure

In this encoding procedure, the recursive subroutine SearchTree is called. Figure 4 provides a flowchart showing its function.

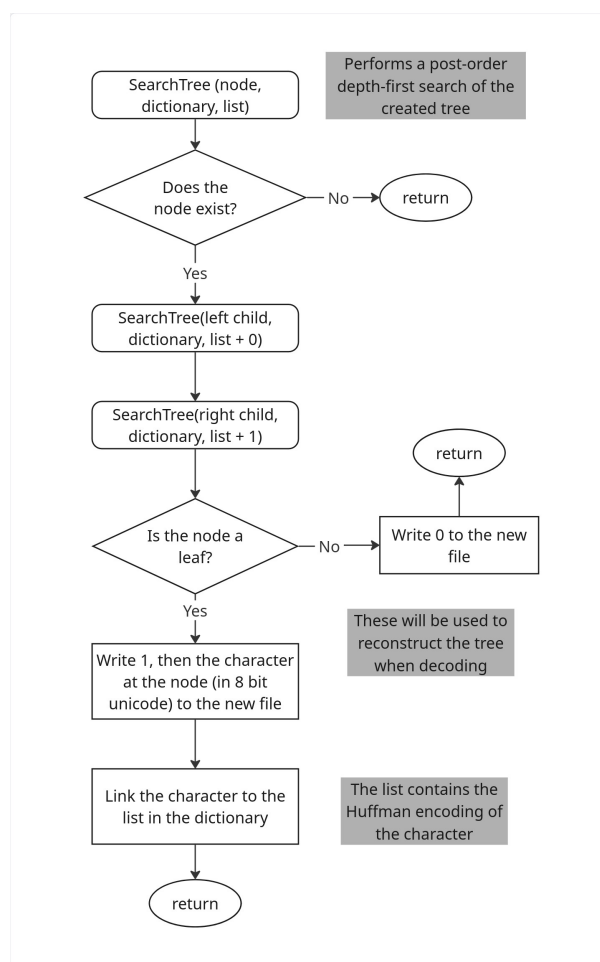


Figure 4: A flowchart depicting my SearchTree procedure

It performs a post-order depth first search of the binary Huffman tree created. By searching the tree in this way, when each node is visited it is written to the compressed file, since the tree needs to be re-read by the decryption program to allow the file to be decrypted. If a leaf node is read, the character belonging to the node is written to the file (in 8-bit Unicode) after a 1 bit. If the node is not a leaf (and so does not contain a character), this is denoted by a 0 being written to the file. This is sufficient for the decoder to fully reconstruct the tree.

The procedure also links each character to its Huffman encoding as the tree is searched (going left in the tree adds a 0 to the encoding, going right adds a 1). This is used later in the program to write each character's Huffman encoding to the compressed file. Filling the dictionary in this procedure means that another search of the tree is not needed, making the code quicker to run.

The use of recursion helps increase the clarity of the procedure and simplify the problem, by breaking it down into smaller sub-problems, until the trivial base case of an empty tree is reached.

The code for this procedure can be found in Listing 1.

```
1 // fill byteMaker with encoded tree, and CharacterEncodings with bits
```

```
encoding characters, uses post order DFS
2 private static void SearchTree(HuffmanNode tree, ByteMaker byteMaker,
  Dictionary<char, List<int>> CharacterEncodings, List<int> current)
3 {
4     if (tree == null) return;
5
6     SearchTree(tree.GetLeft(), byteMaker, CharacterEncodings,
        current.Append(0).ToList());
7     SearchTree(tree.GetRight(), byteMaker, CharacterEncodings,
        current.Append(1).ToList());
8
9     if (tree.HasCharacter()) // is leaf
10    {
11        byteMaker.Add(1);
12        byteMaker.Add(tree.GetCharacter());
13
14        CharacterEncodings[tree.GetCharacter()] = current;
15    }
16    else
17    {
18        byteMaker.Add(0);
19    }
20 }
```

Listing 1: Code for the SearchTree procedure

This tree that is searched in the above procedure is created using a priority queue to continually merge trees, until just one is left. The snippet of the flowchart that relates to this section of the code can be found in Figure 5.

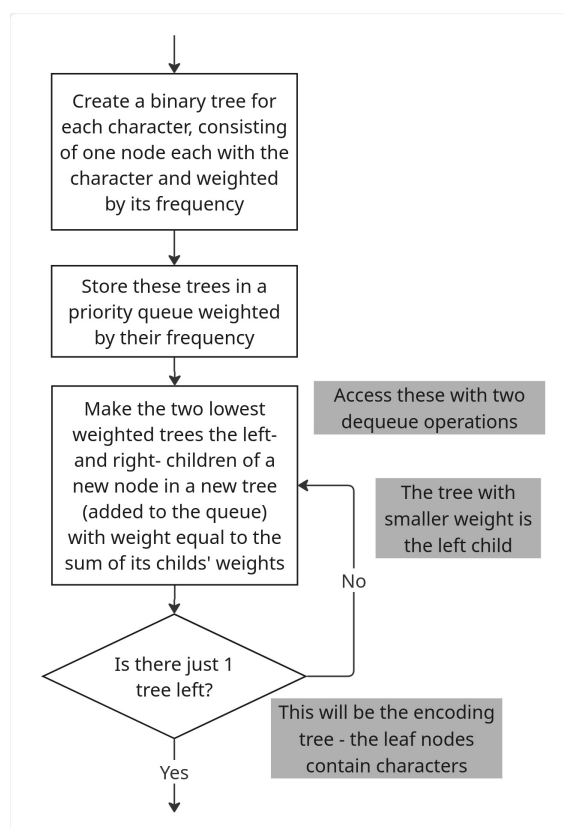


Figure 5: A flowchart depicting the creation of the Huffman tree

First, nodes are created for each character, and these become trees of size 1. They are placed in a priority queue such that the nodes with the greatest frequencies are last. Then, the nodes with the two lowest frequencies are continually dequeued, and made children of a new node, which is enqueued onto the priority queue. Since this is not a leaf node, it does not contain a character. This node's frequency is equal to the sum of the frequencies of its children. The less frequent node is made the left child of the tree. This repeats until just one tree remains, which is the final Huffman tree, which can be used to generate encodings for each character.

The priority queue provides a simple and intuitive way to retrieve the node with the lowest frequency, as desired, without needing to search all of the nodes each time, or intentionally place them into a list in an ordered way.

The code for this section can be found in Listing 2.

```

1 // place characters in priorityqueue (based on freq) to generate the
  tree
2 PriorityQueue<HuffmanNode, int> NodeQueue = new
  PriorityQueue<HuffmanNode, int>(); // "base" trees of size 1
3
4 foreach (KeyValuePair<char, int> kvp in CharacterCounts)
5 {
6     NodeQueue.Enqueue(new HuffmanNode(kvp.Key, kvp.Value), kvp.Value);
7 }
8
9 HuffmanNode tree = MergeNodes(NodeQueue); // tree = top node of tree
  
```

```
10
11 // recursively merge nodes to make a tree
12 private static HuffmanNode MergeNodes(PriorityQueue<HuffmanNode, int>
    trees)
13 {
14     if (trees.Count == 1) return trees.Dequeue();
15
16     HuffmanNode tree1 = trees.Dequeue();
17     HuffmanNode tree2 = trees.Dequeue();
18
19     int newFreq = tree1.GetFrequency() + tree2.GetFrequency();
20
21     HuffmanNode newTree = new HuffmanNode(tree1, tree2, newFreq);
22
23     trees.Enqueue(newTree, newFreq);
24
25     return MergeNodes(trees);
26 }
```

Listing 2: Code for creating the Huffman tree

The implementation here is recursive, with the recursive case passing the new priority queue back to the procedure (to merge again), and the base case being reached when only one tree remains in the priority queue.

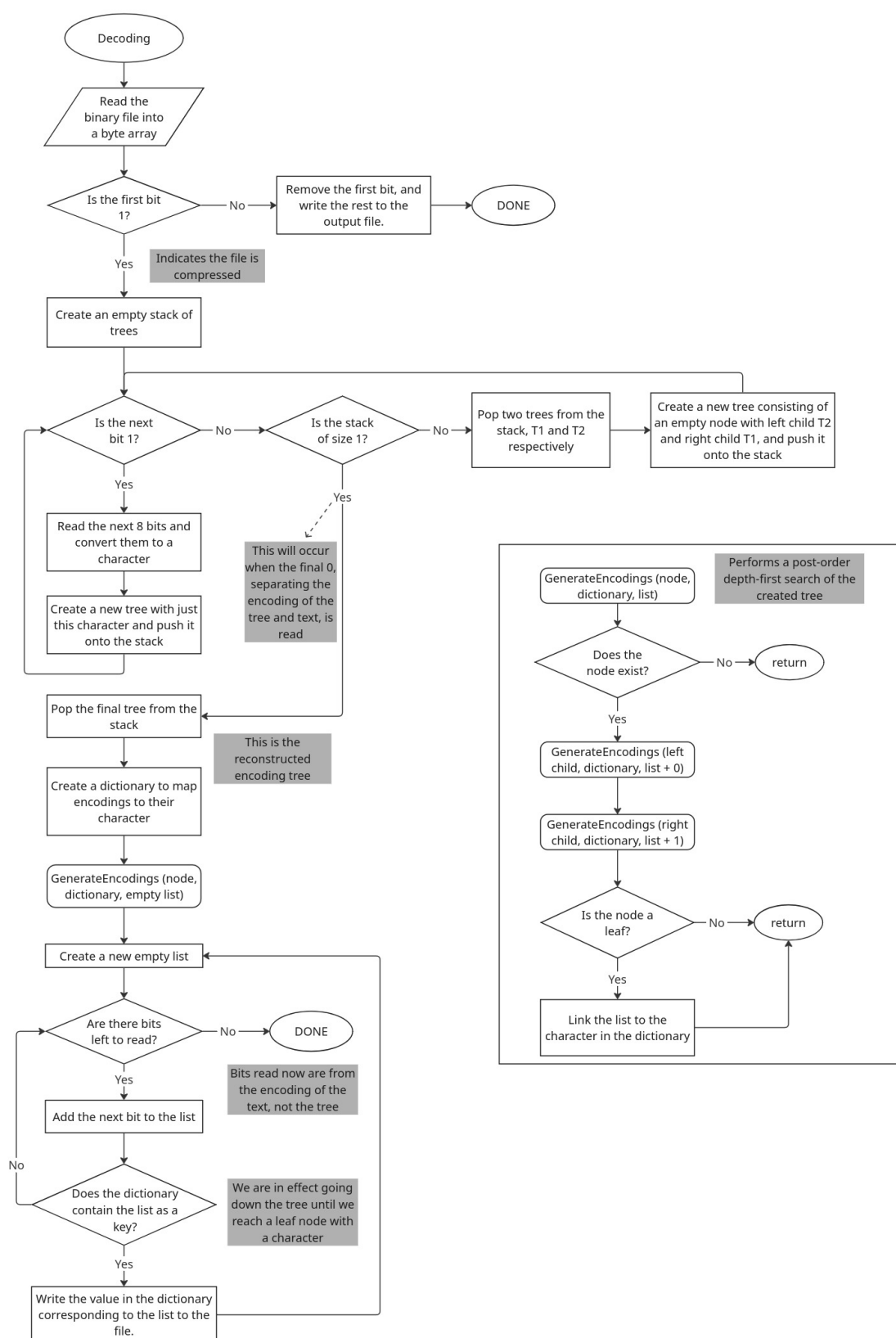


Figure 6: A flowchart depicting my Huffman decoding procedure

In this decoding procedure, the GenerateEncodings procedure recursively calls itself to perform a post-order depth-first search of the Huffman tree to fill a dictionary mapping Huffman encodings to the characters they represent. This works identically to the SearchTree procedure in the encoding procedure, just without writing anything to a file, and with the dictionary direction reversed (in the encoding procedure, characters are linked to their Huffman encoding).

A stack is used to read the Huffman tree representation from the file, and store it in the program as a tree. Figure 7 is a snippet of the main decoding flowchart that relates to this.

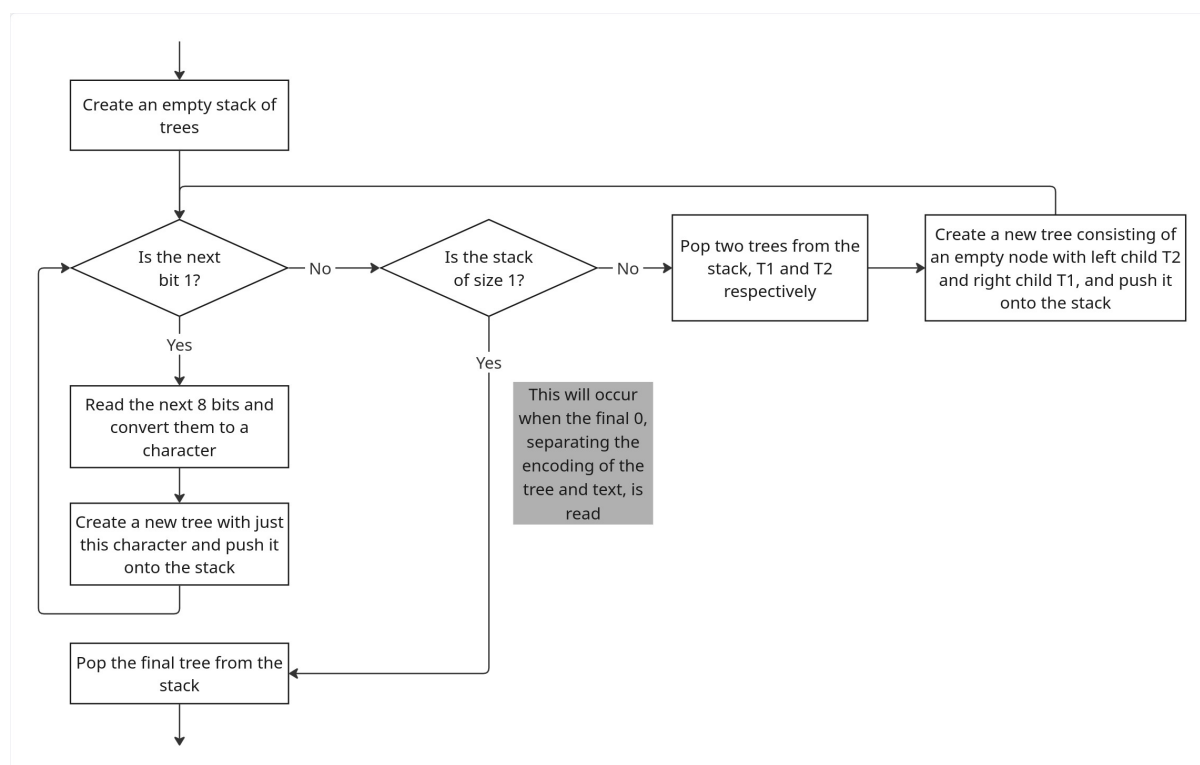


Figure 7: A flowchart depicting the reconstruction of the Huffman tree

The start of the compressed file contains an encoding of the Huffman tree used. This was created, and written, in the encoding procedure, with a post-order depth-first tree search. Here, the tree is reconstructed.

If a 1 bit is read - indicating a character comes next - the next 8 bits are read and converted into a Unicode character, then pushed onto the stack as a new node (which is in effect a tree of size 1).

Otherwise, if a 0 bit is read, one of two options occurs. If the stack is of size 1, this indicates that the zero read represented the end of the tree encoding, and the program pops the one tree in the stack, and uses that as the Huffman tree to decode the text in the file. Otherwise, the 0 indicates a non-leaf node, so the top two trees are popped from the stack, and made children of a new node which is pushed back onto the stack. The first tree popped becomes the right child.

The code for this algorithm can be found in Listing 3.

```

1 Stack<HuffmanNode> NodeStack = new Stack<HuffmanNode>();
2
3 while (true)
4 {
5     int nextBit = byteReader.Read();
6
7     if (nextBit == 1)
8     {
9         char? nextChar = byteReader.ReadChar();
10        if (nextChar == null) break;
11
12        NodeStack.Push(new HuffmanNode((char)nextChar));
13    }
14    else
15    {
16        if (NodeStack.Count() == 1) break;
17
18        HuffmanNode right = NodeStack.Pop();
19        HuffmanNode left = NodeStack.Pop();
20
21        NodeStack.Push(new HuffmanNode(left, right));
22    }
23 }
24
25 HuffmanNode tree = NodeStack.Pop(); // there should be just one node
    left

```

Listing 3: Code for reconstructing the Huffman tree

To represent the binary Huffman tree, I made a dedicated HuffmanNode class. Each node is linked to any children it has, so a tree is just represented as its top-node. Leaf nodes also store what character they represent. This class shows polymorphism by having two constructors taking different arguments due to the different information available when making a leaf and non-leaf node.

The pseudocode for this can be found in Listing 4, and the code in Listing 5.

```

1 CLASS HuffmanNode
2     PRIVATE CHARACTER _character
3     PRIVATE INTEGER _frequency
4     PRIVATE HuffmanNode _left
5     PRIVATE HuffmanNode _right
6
7     PRIVATE BOOLEAN _hasCharacter
8
9     PUBLIC CONSTRUCTOR HuffmanNode (CHARACTER char, INTEGER
        frequency)
10         _character <- char
11         _frequency <- frequency
12
13         _hasCharacter <- TRUE

```

```

14      ENDCONSTRUCTOR
15
16      PUBLIC CONSTRUCTOR HuffmanNode (HuffmanNode left, HuffmanNode
17          right, INTEGER frequency)
18          _left <- left
19          _right <- right
20          _frequency <- frequency
21
22          _hasCharacter <- FALSE
23      ENDCONSTRUCTOR
24
25      PUBLIC SUBROUTINE GetFrequency()
26          RETURN _frequency
27      ENDSUBROUTINE
28
29      PUBLIC SUBROUTINE HuffmanNode GetLeft()
30          RETURN _left
31      ENDSUBROUTINE
32
33      PUBLIC SUBROUTINE HuffmanNode GetRight()
34          RETURN _right
35      ENDSUBROUTINE
36
37      PUBLIC SUBROUTINE bool HasCharacter()
38          RETURN _hasCharacter
39      ENDSUBROUTINE
40
41      PUBLIC SUBROUTINE char GetCharacter()
42          RETURN _character
43      ENDSUBROUTINE
44  ENDClass

```

Listing 4: Pseudocode for the HuffmanNode class

```

1  class HuffmanNode
2  {
3      private char _character;
4      private int _frequency;
5      private HuffmanNode? _left;
6      private HuffmanNode? _right;
7
8      private bool _hasCharacter; // "isLeaf"
9
10     public HuffmanNode(char character, int frequency = 1)
11     {
12         _character = character;
13         _frequency = frequency;
14
15         _hasCharacter = true;
16     }
17

```

```

18     public HuffmanNode(HuffmanNode left, HuffmanNode right, int
        frequency = 1)
19     {
20         _left = left;
21         _right = right;
22         _frequency = frequency;
23
24         _hasCharacter = false;
25     }
26
27     public int GetFrequency() => _frequency;
28
29     public HuffmanNode? GetLeft() => _left;
30     public HuffmanNode? GetRight() => _right;
31
32     public bool HasCharacter() => _hasCharacter;
33     public char GetCharacter() => _character;
34 }
35
36 // NOTE: tree is just a "top-node" - links to left/right

```

Listing 5: Code for the HuffmanNode class

2.1.2 Reading and Writing Bytes

These flowchart implementations above assumed that writing bits to, and reading bits from, files is a trivial task. However, this proved to be quite challenging, since one can only write to, and read from, files one byte at a time in C#. Therefore, I needed to implement additional helper methods for this.

I constructed pseudocode for ByteMaker (Listing 6) and ByteReader (Listing 7) classes, which respectively allow writing to and reading from binary files.

```

1 CLASS ByteMaker
2     PRIVATE INTEGER ARRAY _bits
3     PRIVATE INTEGER _bitIndex
4     PRIVATE INTEGER _trailingZeros
5
6     PRIVATE BYTE LIST _bytes
7
8     PUBLIC CONSTRUCTOR ByteMaker (BOOLEAN isCompressed)
9         _bitIndex <- 4
10        IF (isCompressed = TRUE) THEN
11            _bits <- [1, 0, 0, 0, 0, 0, 0, 0, 0]
12        ELSE
13            _bits <- [0, 0, 0, 0, 0, 0, 0, 0, 0]
14        ENDIF
15    ENDCONSTRUCTOR
16
17    PUBLIC SUBROUTINE Add (INTEGER bit)

```

```

18         IF (bit <> 1 AND bit <> 0) THEN
19             RETURN
20         ENDIF
21
22         _bits[_bitIndex] <- bit
23         _bitIndex <- _bitIndex + 1
24
25         IF (_bitIndex = 8) THEN
26             updateByte()
27         ENDIF
28     ENDSUBROUTINE
29
30     PUBLIC SUBROUTINE Add (INTEGER LIST bits)
31         FOR (INTEGER bit IN bits)
32             Add(bit)
33         ENDFOR
34     ENDSUBROUTINE
35
36     PUBLIC SUBROUTINE Add (CHARACTER char)
37         BYTE charByte <- char TO BYTE
38         FOR (INTEGER i <- 7 TO 0 STEP -1)
39             INTEGER bit <- (RIGHTSHIFT charByte i) AND 1
40
41             Add(bit)
42         ENDFOR
43     ENDSUBROUTINE
44
45     PUBLIC SUBROUTINE Export()
46         updateByte()
47
48         BYTE trailingZerosByte <- _trailingZeros TO BYTE
49         _bytes[0] <- _bytes[0] OR (LEFTSHIFT trailingZerosByte
50             4)
51
52         RETURN (_bytes TO ARRAY)
53     ENDSUBROUTINE
54
55     PRIVATE SUBROUTINE updateByte()
56         BYTE newByte
57         FOR (INTEGER i <- 0 TO 7 STEP 1)
58             BYTE mergeByte <- (LEFTSHIFT _bits[i] 7-i) TO
59                 BYTE
60             newByte <- newByte OR mergeByte
61         ENDFOR
62         ADD newByte TO _bytes
63
64         _bits <- [0, 0, 0, 0, 0, 0, 0, 0]
65         _trailingZeros <- 8 - _bitIndex
66         _bitIndex <- 0
67     ENDSUBROUTINE
68 ENDCLASS

```

Listing 6: Pseudocode for the ByteMaker class

The ByteMaker provides methods to write single bits (as integers 0/1), lists of bits, and characters to a private list of bytes, which it can "export" upon completion, returning a byte array that can be written to the binary file. The class holds an 8 long array of integers to represent the current byte, and this is added to the byte array when it is filled (when the index used to write to it reaches 8). This is done by converting it into a byte using bitwise operations (shifting each bit to its position, then performing an OR operation with the byte being constructed).

When all desired bits have been written to the file, it is likely that this is not at the end of a byte, and so the remaining bits are left as 0, since you can only write to files in bytes. However, the number of such bits need to be stored in some form, so it is clear which bits to ignore when decoding. Therefore, bits 1-3 of the first byte are kept clear for writing a 3-bit integer between 0-7 (the possible number of trailing bits) at the end. Note that bit 0 is used to indicate whether the file is being compressed in the first place. This is why the index begins at 4 rather than 0 for the first bit.

```

1 CLASS ByteReader
2     PRIVATE BYTE LIST _bytes
3     PRIVATE INTEGER _bitIndex
4     PRIVATE INTEGER _trailingNum
5
6     PUBLIC CONSTRUCTOR ByteReader (BYTE ARRAY bytes)
7         _bytes <- bytes TO LIST
8         _bitIndex <- 0
9
10        INTEGER firstBit <- Read()
11        INTEGER a <- Read()
12        INTEGER b <- Read()
13        INTEGER c <- Read()
14        _trailingNum <- 4*a + 2*b + c
15
16        IF (firstBit = 0) THEN
17            RETURN FALSE
18        ELSE
19            RETURN TRUE
20        ENDIF
21    ENDCONSTRUCTOR
22
23    PUBLIC SUBROUTINE Read()
24        IF (LENGTH _bytes = 0) THEN
25            RETURN -1
26        ENDIF
27        IF (LENGTH _bytes = 1 AND _bitIndex >= 8 -
28            _trailingNum) THEN
29            RETURN -1
30        ENDIF

```

```

31         INTEGER bit <- (RIGHTSHIFT _bytes[0] (7 - _bitIndex))
           AND 1
32         _bitIndex <- _bitIndex + 1
33
34         RETURN bit
35     ENDSUBROUTINE
36
37     PUBLIC SUBROUTINE ReadChar()
38         BYTE nextByte <- 0
39
40         FOR (INTEGER i <- 0 TO 7 STEP 1)
41             INTEGER bit <- Read()
42             IF (bit = -1) THEN
43                 RETURN ' '
44             ELSE IF (bit = 1) THEN
45                 nextByte <- nextByte OR (LEFTSHIFT 1
                                         (7-i))
46             ENDIF
47         ENDFOR
48
49         RETURN (nextByte TO CHARACTER)
50     ENDSUBROUTINE
51
52     PRIVATE SUBROUTINE Update()
53         IF (_bitIndex <> 8) THEN
54             RETURN
55         ENDIF
56
57         REMOVE INDEX 0 FROM _bytes
58         _bitIndex <- 0
59     ENDSUBROUTINE
60 ENDCLASS

```

Listing 7: Pseudocode for the ByteReader class

This ByteReader class acts as the opposite of the ByteMaker: it takes in a byte array when being instantiated, and provides methods for reading the next bit and character (8 bits). The constructor returns a boolean value indicating whether the first bit is 0 or 1, and so whether compression was used on the file. It also uses the next 3 bits to determine the number of trailing 0s at the end of the file, so they can be ignored when reading. This is indicated by the Read method returning -1 once those characters have been reached.

The Read method uses bitwise operations to get the bit at a certain index from the byte at index 0 of the list. Index 0 is always used as the Update method removes bytes that have already been fully read from the start of the list. The right shift operation places the desired bit at the end of the byte, and performing an AND with 1 returns the value of the last bit of the right-shifted byte, as desired.

The code for the ByteMaker and ByteReader classes can be found in Listings 8 and 9 respectively.

```

1 class ByteMaker

```



```

2 {
3     private int[] _bits;
4     private int _bitIndex;
5     private int _trailingZeros;
6
7     private List<byte> _bytes;
8
9     public ByteMaker(bool isCompressed)
10    {
11        _bits = [isCompressed ? 1 : 0, 0, 0, 0, 0, 0, 0, 0]; // length
12                      8
13        // first bit indicates compressed, next 3 bits for trailing
14                      zeros
15        _bitIndex = 4;
16        _trailingZeros = 0;
17
18        _bytes = new List<byte>();
19    }
20
21    public void Add(int bit)
22    {
23        if (bit != 1 && bit != 0) return;
24
25        _bits[_bitIndex++] = bit;
26
27        if (_bitIndex == 8) updateByte();
28    }
29
30    public void Add(List<int> bits)
31    {
32        foreach (int bit in bits) Add(bit);
33    }
34
35    public void Add(char character)
36    {
37        byte characterByte = (byte)character;
38
39        for (int i = 7; i >= 0; i--)
40        {
41            // right-shift, then AND with 1 to get "last" bit
42            int bit = (characterByte >> i) & 1;
43
44            Add(bit);
45        }
46    }
47
48    public byte[] Export()
49    {
50        updateByte();
51
52        // push to bits 1-4, OR since already 0

```

```

51     _bytes[0] |= (byte)(_trailingZeros << 4);
52
53     return _bytes.ToArray();
54 }
55
56 private void updateByte()
57 {
58     byte newByte = new byte();
59
60     for (int i = 0; i < 8; i++)
61     {
62         // left-shift, then OR with existing byte
63         newByte |= (byte)(_bits[i] << (7-i));
64     }
65
66     _bytes.Add(newByte);
67
68     _bits = [0, 0, 0, 0, 0, 0, 0, 0]; // length 8
69     _trailingZeros = 8 - _bitIndex;
70     _bitIndex = 0;
71 }
72 }

```

Listing 8: Code for the ByteMaker class

```

1 class ByteReader
2 {
3     private List<byte> _bytes;
4     private int _bitIndex;
5     private int _trailingNum;
6
7     public ByteReader(byte[] bytes, out bool compressed)
8     {
9         _bytes = bytes.ToList();
10        _bitIndex = 0;
11
12        int firstBit = Read();
13
14        if (firstBit == 0) compressed = false;
15        else compressed = true;
16
17        int a = Read();
18        int b = Read();
19        int c = Read();
20
21        _trailingNum = a*4 + b*2 + c;
22    }
23
24    public int Read()
25    {
26        if (_bytes.Count() == 0) return -1;

```

```

27
28     // don't read padding zeros
29     if (_bytes.Count() == 1 && _bitIndex >= 8 - _trailingNum)
30         return -1;
31
32     int bit = (_bytes[0] >> (7 - _bitIndex++)) & 1;
33
34     Update();
35
36     return bit;
37 }
38
39 public char? ReadChar()
40 {
41     byte nextByte = 0;
42
43     for (int i = 0; i < 8; i++)
44     {
45         int bit = Read();
46         if (bit == -1) return null;
47
48         // shift 1 to its position and OR with existing byte
49         if (bit == 1) nextByte |= (byte)(1 << (7-i));
50     }
51
52     return (char)nextByte;
53 }
54
55 private void Update()
56 {
57     if (_bitIndex != 8) return;
58
59     _bytes.RemoveAt(0);
60     _bitIndex = 0;
61 }

```

Listing 9: Code for the ByteReader class

2.1.3 UML Class Diagram

Figure 8 shows a UML class diagram depicting my design for this component of the program. No inheritance, composition, or aggregation is used, but the HuffmanNode constructor and the ByteMaker Add method show polymorphism by having overloads.

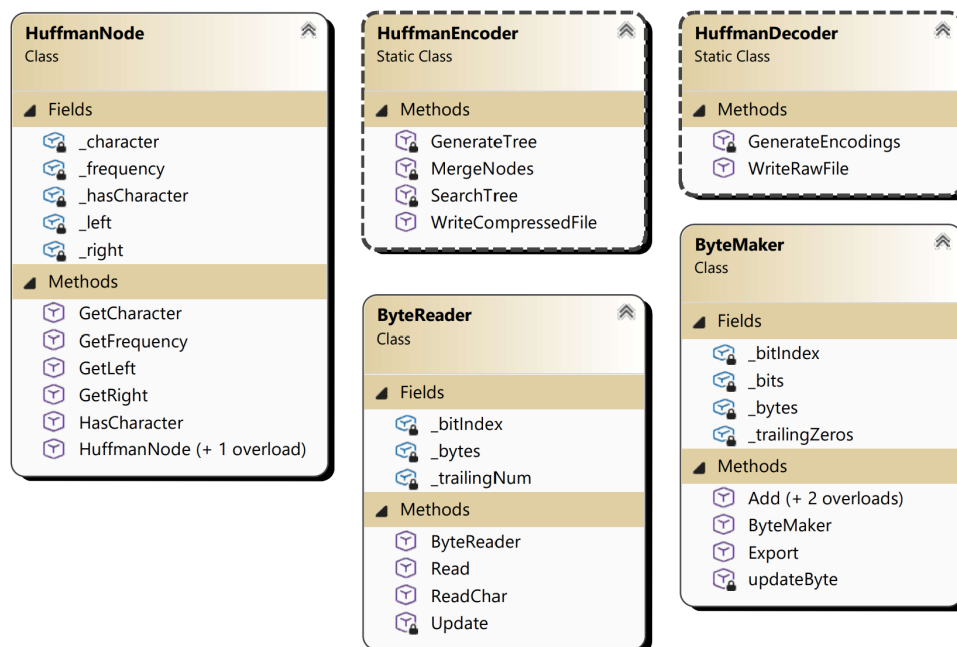


Figure 8: A UML class diagram for the compression program

2.2 RSA Key Exchange

In my Analysis (Section 1.7.1), I decided that I will use the RSA algorithm for key exchange. This asymmetric encryption algorithm will be used to share a symmetric key between the client and server, to be used for any further communications between the devices.

From the Analysis, the following procedure can be followed to generate RSA keys:

1. The receiver generates two large primes, p, q , then computes their product, $n = pq$.
2. Euler's Totient function, $\phi(pq) = (p - 1)(q - 1)$, is applied to n .
3. An encryption exponent, e , is selected such that $1 < e < \phi(n)$, and e is relatively prime with $\phi(n)$. Note that $a, b \in \mathbb{Z}^+$ are relatively prime $\iff \nexists x \in \mathbb{Z} : x > 1, x | a, \text{ and } x | b$.
4. A decryption exponent, d , is calculated such that $de = 1 \pmod{\phi(n)}$. The extended Euclidean algorithm provides a method for this.
5. The public key is formed from (n, e) and the private key (n, d) . The receiver sends their public key only to the sender.
6. The encrypted ciphertext, C , is calculated through $C = M^e \pmod{n}$, where M is some numerical representation of the message.
7. The ciphertext is decrypted into the plaintext message, M , by the calculation $M = C^d \pmod{n}$.

The first prerequisite to implementing the RSA algorithm is a method to generate very large (512 bit) prime numbers. Since it is intractable to generate random very large prime numbers that are guaranteed to be prime, the Miller-Rabin primality test is very often used. This is an algorithm that tests whether a large prime number is probably prime. Wikipedia provides a description of this algorithm [6].

For each base tested, if this algorithm labels an integer composite, it must be composite, but it has a $\frac{1}{4}$ probability of incorrectly labelling the integer prime. However, this means that running the algorithm n times (with n bases) reduces this probability of incorrectly labelling a composite number prime every time (since if it ever labels it composite then it must be composite) to 4^{-n} . Running this 40 times, the probability of it giving a wrong result is 4^{-40} , which is exceedingly unlikely.

Figure 9 displays a flowchart demonstrating the steps this algorithm takes.

Listing 10 contains the code for this algorithm.

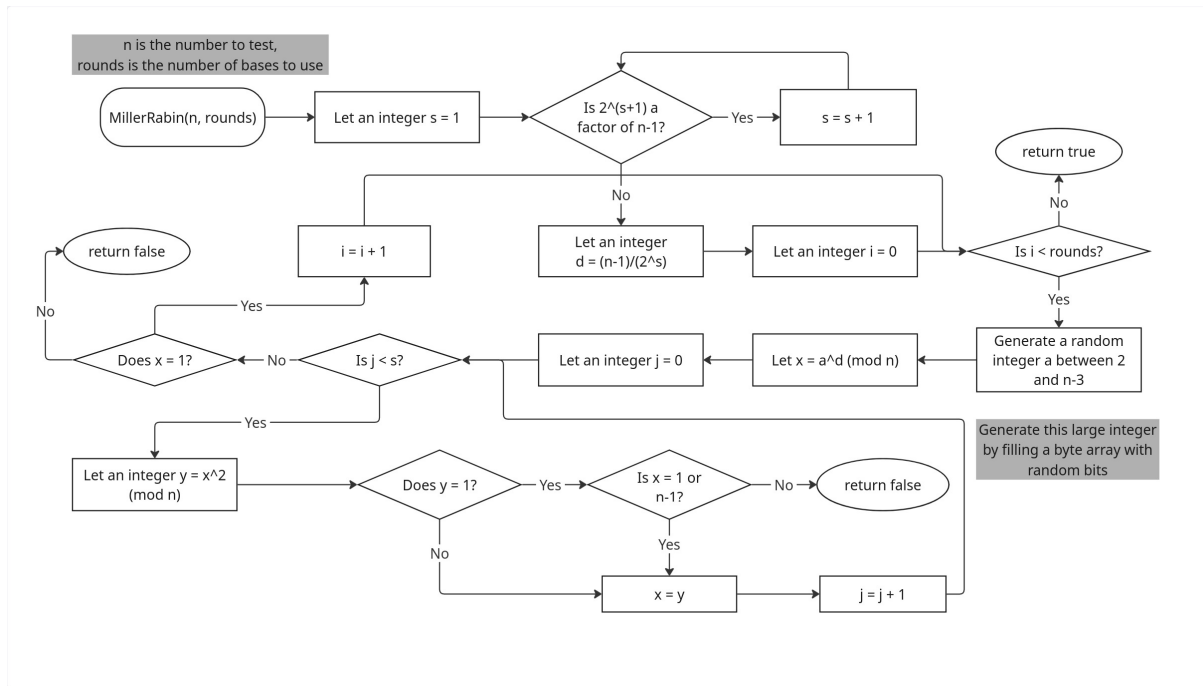


Figure 9: A flowchart for the Miller-Rabin algorithm

```

1 private static bool MillerRabin(BigInteger n, int rounds)
2 {
3     int s = 1;
4     while ((n-1) % (BigInteger)Math.Pow(2, s+1) == 0) s++;
5
6     BigInteger d = (n-1)/(BigInteger)(Math.Pow(2,s));
7
8     for (int i = 0; i < rounds; i++)
9     {
10         BigInteger a = GenerateInRange(2, n-3);
11
12         BigInteger x = BigInteger.ModPow(a, d, n);
13
14         for (int j = 0; j < s; j++)
15         {
16             BigInteger y = BigInteger.ModPow(x, 2, n);
17
18             if (y == 1 && x != 1 && x != n-1) return false;
19
20             x = y;
21         }
22
23         if (x != 1) return false;
24     }
25
26     return true;
27 }

```

Listing 10: Code for the Miller-Rabin algorithm

This is used to generate large primes by continually testing randomly generated primes until one passes the test. Large primes (512 bit in this case) are generated by filling a byte array (64 long in this case) with random bits, ensuring the most significant bit is 1, so the prime is actually large enough, and the least significant bit is also 1, so the number is not a multiple of two (since they are clearly not prime). A check is also performed to ensure that for the composite n formed by $n = pq$, where p, q are primes being generated here, $\phi(n)$ is relatively prime with the encryption exponent e , which is chosen as 65537 in this case, for its favourable properties.

A flowchart for this procedure can be found in Figure 10, and code for it in Listing 11.

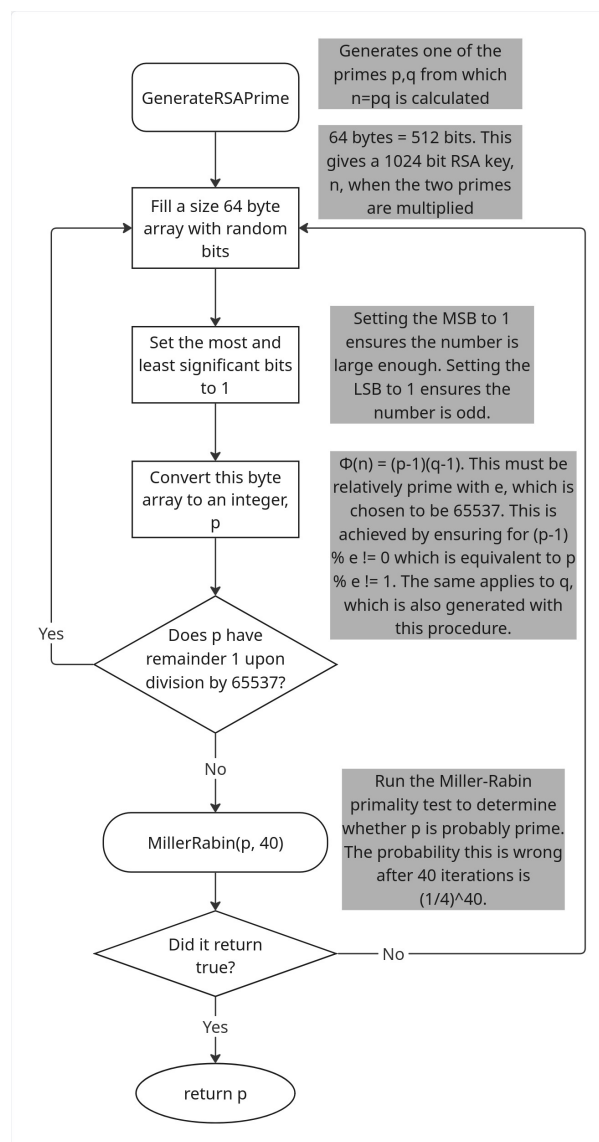


Figure 10: A flowchart for the procedure to generate an RSA prime

```

1 private static BigInteger GenerateRSAPrime(BigInteger e) // e must be
   prime in this implementation!
2 {

```

```

3      while (true)
4      {
5          BigInteger bigInt;
6
7          do bigInt = GenerateOdd512BitInt();
8          while (bigInt % e == 1); // ensures e will be relatively prime
                                   with phi(n), as e used is prime
9
10         if (MillerRabin(bigInt, 40)) return bigInt;
11     }
12 }
13
14 private static BigInteger GenerateOdd512BitInt()
15 {
16     byte[] randomInt = new byte[64]; // 512 bits = 64 bytes
17     RandomNumberGenerator.Fill(randomInt);
18
19     randomInt[0] |= (byte)(128); // sets MSB to 1, so num is 512 bits
20
21     randomInt[63] |= (byte)(1); // sets LSB to 1 - odd
22
23     return new BigInteger(randomInt, true, true); // isUnsigned,
                                   isBigEndian
24 }

```

Listing 11: Code for the procedure to generate an RSA prime

The final major algorithm needed in the implementation of RSA is the Extended Euclidean Algorithm, which is used to generate the decryption exponent, d , such that $de = 1 \pmod{\phi(n)}$. Wikipedia provides a mathematical description of this algorithm [3]. Figure 11 provides a flowchart representing the algorithm for this, and Listing 12 shows the code for this procedure.

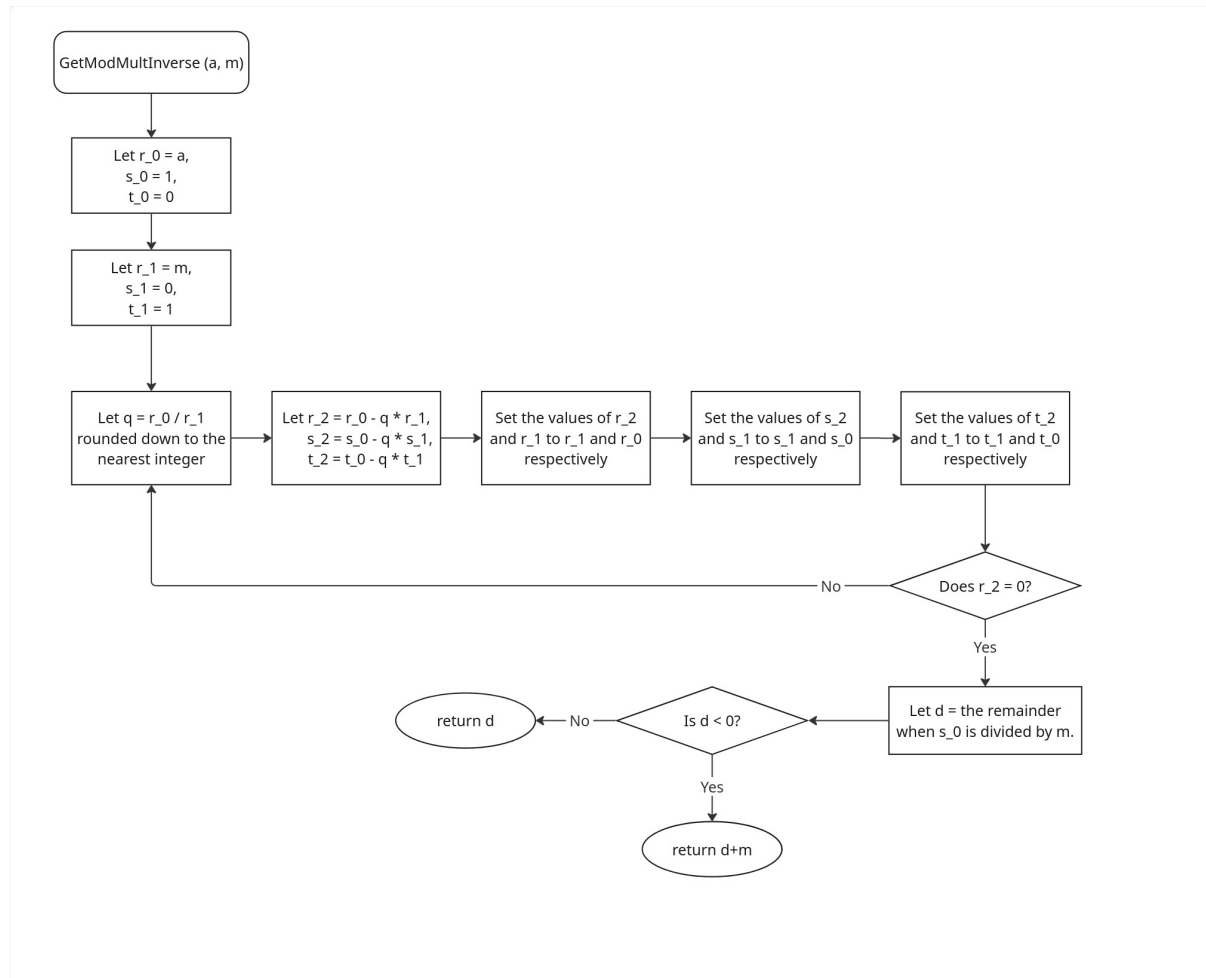


Figure 11: A flowchart for the Extended Euclidean Algorithm

```

1 private static BigInteger GetModMultInverse(BigInteger a, BigInteger
  m) // uses the extended euclidean algorithm to find x s.t. ax = 1
  (mod m)
2 {
3     BigInteger r_0 = a;
4     BigInteger s_0 = 1;
5     BigInteger t_0 = 0;
6
7     BigInteger r_1 = m;
8     BigInteger s_1 = 0;
9     BigInteger t_1 = 1;
10
11     BigInteger r_2, s_2, t_2;
12
13     do
14     {
15         BigInteger q = r_0 / r_1;
16
17         r_2 = r_0 - q * r_1;
18         s_2 = s_0 - q * s_1;

```

```

19     t_2 = t_0 - q * t_1;
20
21     (r_0, r_1) = (r_1, r_2);
22     (s_0, s_1) = (s_1, s_2);
23     (t_0, t_1) = (t_1, t_2);
24
25 } while (r_2 != 0);
26
27 BigInteger d = s_0 % m;
28 if (d < 0) d += m;
29
30 return d;
31 }

```

Listing 12: Code for the Extended Euclidean Algorithm

All that is left now is to put this all together, and generate the public and private RSA keys. The public key consists of (n, e) , and the private key (n, d) , so a procedure is needed to generate the tuple (n, e, d) . Figure 12 provides a flowchart for this, and my implementation can be seen in Listing 13.

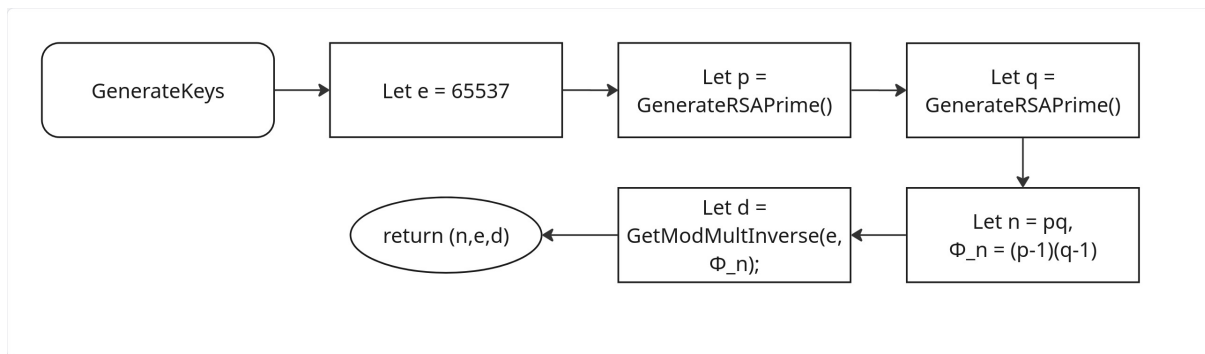


Figure 12: A flowchart for the procedure to generate RSA keys

```

1 public static Tuple<BigInteger, BigInteger, BigInteger> GenerateKeys()
2 {
3     BigInteger e = 65537;
4
5     BigInteger p = RSA.GenerateRSAPrime(e);
6     BigInteger q = RSA.GenerateRSAPrime(e);
7
8     BigInteger n = p*q;
9     BigInteger phin = (p-1)*(q-1);
10
11     BigInteger d = RSA.GetModMultInverse(e, phin);
12
13     return new Tuple<BigInteger, BigInteger, BigInteger>(n,e,d);
14 }

```

Listing 13: Code for the procedure to generate RSA keys

2.3 Verifying Communication Integrity

To verify the integrity of the files transmitted, as described in the Analysis (Section 1.7.3), I will compute a hash of the data being sent, and transmit the hash of the message before the rest of the message. The recipient can then compute the hash of the message itself, and compare that to the hash it received. If they match, then they can be confident that the message was not corrupted in transmission.

As decided in the Analysis, I will use the popular SHA-256 algorithm for this, which generates a fixed size 256-bit hash. Listing 14 contains pseudocode for the procedures to generate a hash, and to verify one.

```

1 PUBLIC SUBROUTINE Hash (BYTE ARRAY messageBytes)
2     BYTE ARRAY hashBytes
3
4     hashBytes <- GET HASH OF messageBytes
5
6     RETURN hashBytes
7 ENDSUBROUTINE
8
9 PUBLIC SUBROUTINE VerifyHash (BYTE ARRAY messageBytes, BYTE ARRAY
    hashBytes)
10     BYTE ARRAY messageHash <- Hash(messageBytes)
11
12     FOR (INTEGER i <- 0 TO (LENGTH OF messageHash - 1) STEP 1)
13         IF (messageHash[i] <> hashBytes[i]) THEN
14             RETURN FALSE
15         ENDIF
16     ENDFOR
17
18     RETURN TRUE
19 ENDSUBROUTINE

```

Listing 14: Pseudocode for the hashing procedures

The verification subroutine checks each byte in the byte arrays representing the hash computed and hash received, and returns false if any do not match exactly. Code for this can be found in Listing 15.

```

1 using System.Security.Cryptography;
2
3 static class Hashing
4 {
5     public static byte[] Hash(byte[] messageBytes)
6     {
7         byte[] hashBytes;
8
9         using (SHA256 HashMaker = SHA256.Create())
10        {
11            hashBytes = HashMaker.ComputeHash(messageBytes);
12        }
13    }
14 }

```

```

13
14     return hashBytes;
15 }
16
17 public static bool VerifyHash(byte[] messageBytes, byte[] hash)
18 {
19     byte[] originalHashed = Hash(messageBytes);
20
21     if (originalHashed.Length != hash.Length) return false;
22
23     for (int i = 0; i < hash.Length; i++)
24     {
25         if (originalHashed[i] != hash[i]) return false;
26     }
27
28     return true;
29 }
30 }

```

Listing 15: Code for the hashing procedures

2.4 Transmission Procedures

In order to send messages through a network stream, at the simplest level, there must be a method for the recipient to know how long the data being sent is. I can achieve this by encoding the number of bytes to expect in the first 4 bytes of each transmission. As such, the receiver knows to read the first 4 bytes (which will be an integer n), read the next n bytes in whatever format it is expecting, and then repeat this procedure for the rest of the stream, knowing that it is a new transmission.

This requires some serialisation and "de-serialisation" subroutines to handle adding the length of the message to the start of it, and reading exactly as many bytes as are indicated. Listing 16 provides pseudocode for these methods, and Listing 17 provides the code for it.

```

1 CLASS Transmission
2     PUBLIC SUBROUTINE Serialise (BYTE ARRAY bytes)
3         BYTE ARRAY[4] bytesLengthBytes <- (LENGTH OF bytes) TO
           BYTE ARRAY
4
5         BYTE ARRAY data <- bytesLengthBytes + bytes
6
7         RETURN data
8     ENDSUBROUTINE
9
10    PUBLIC SUBROUTINE Serialise (BIG INTEGER bigInt)
11        BYTE ARRAY bigIntBytes <- bigInt TO BYTE ARRAY
12
13        RETURN Serialise(bigIntBytes)
14    ENDSUBROUTINE

```

```

15
16     PUBLIC SUBROUTINE Serialise (STRING data)
17         RETURN Serialise(data TO BYTE ARRAY)
18     ENDSUBROUTINE
19
20     PUBLIC SUBROUTINE ReadByteArray (NETWORK STREAM stream)
21         INTEGER bytesLength <- ReadBytes(stream, 4) TO INTEGER
22
23         RETURN ReadBytes(stream, bytesLength)
24     ENDSUBROUTINE
25
26     PUBLIC SUBROUTINE ReadBigInt (NETWORK STREAM stream)
27         RETURN ReadByteArray(stream) TO BIG INTEGER
28     ENDSUBROUTINE
29
30     public static byte[] ReadBytes(Stream stream, int length)
31         BYTE ARRAY[length] bytes
32
33         bytes <- READ length FROM stream
34
35         RETURN bytes
36     ENDSUBROUTINE
37 ENDCLASS

```

Listing 16: Pseudocode for the serialisation procedures

```

1  using System.Numerics;
2  using System.Net.Sockets;
3
4  static class Transmission
5  {
6      public static byte[] Serialise(byte[] bytes)
7      {
8          byte[] bytesLengthBytes = BitConverter.GetBytes(bytes.Length);
9          // length 4 always
10
11          byte[] data =
12              bytesLengthBytes.ToList().Concat(bytes.ToList()).ToArray();
13
14          return data;
15      }
16
17      public static byte[] Serialise(BigInteger bigInt)
18      {
19          byte[] bigIntBytes = bigInt.ToByteArray(true, true); //
20              isUnsigned, isBigEndian
21
22          return Serialise(bigIntBytes);
23      }
24
25      public static byte[] Serialise(string data) =>

```

```

23         Serialise(System.Text.Encoding.ASCII.GetBytes(data));
24
25     public static BigInteger ReadBigInt(NetworkStream stream) => new
26         BigInteger(ReadByteArray(stream), true, true);
27
28     public static byte[] ReadByteArray(Stream stream)
29     {
30         int bytesLength = BitConverter.ToInt32(ReadBytes(stream, 4));
31
32         return ReadBytes(stream, bytesLength);
33     }
34
35     public static byte[] ReadBytes(Stream stream, int length)
36     {
37         byte[] bytes = new byte[length];
38
39         int read = 0;
40         while (read < length) read += stream.Read(bytes, read, length
41             - read);
42
43         return bytes;
44     }
45 }

```

Listing 17: Code for the serialisation procedures

Having established an encrypted AES stream (using the `CryptoStream` class in C#), some helper methods are needed to send and receive various data types. I designed `FileSender` and `FileReceiver` classes for this purpose. These both inherit from an abstract `Transferer` class, which stores the common `CryptoStream` attribute, and a common constructor for the two derived classes. Using classes, rather than just procedures, is beneficial here since, as well as grouping together the related subroutines cleanly, the `CryptoStream` can be passed in just once upon instantiation of an object. It is important that this is not re-created, since that would interrupt the communication stream. Pseudocode for these classes can be found in Listing 18, and the code in Listing 19.

```

1  ABSTRACT CLASS Transferer
2      PROTECTED CRYPTO STREAM _cryptoStream
3
4      PUBLIC CONSTRUCTOR Transferer (NETWORK STREAM stream, AES
5          ENCRYPTOR aes, CRYPTO STREAM MODE mode)
6          SET aes PADDING MODE TO ZEROS
7
8          IF (mode = WRITE) THEN
9              _cryptoStream <- NEW CRYPTO STREAM(stream, aes
10                  ENCRYPTOR, mode, LEAVE OPEN)
11          ELSE
12              _cryptoStream <- NEW CRYPTO STREAM(stream, aes
13                  DECRYPTOR, mode, LEAVE OPEN)
14          ENDIF
15      ENDCONSTRUCTOR

```

```

13  ENDCLASS
14
15  CLASS FileSender FROM Transferer
16      PUBLIC CONSTRUCTOR FileSender (NETWORK STREAM stream, AES
17          ENCRYPTOR aes)
18          CALL BASE(stream, aes, WRITE)
19      ENDCONSTRUCTOR
20
21      PUBLIC SUBROUTINE SendFile (BYTE ARRAY fileBytes, STRING
22          fileName)
23          BYTE ARRAY hashBytes <- Hash(fileBytes)
24
25          WRITE Serialise(hashBytes) TO _cryptoStream
26
27          WRITE Serialise(fileName) TO _cryptoStream
28
29          SendData(fileBytes)
30      ENDSUBROUTINE
31
32      PUBLIC SUBROUTINE SendData (STRING data)
33          BYTE ARRAY bytes <- data TO BYTE ARRAY
34
35          BYTE ARRAY hashBytes <- HASH(bytes)
36
37          WRITE Serialise(hashBytes) TO _cryptoStream
38
39          SendData(bytes)
40      ENDSUBROUTINE
41
42      PUBLIC SUBROUTINE SendInt (INTEGER data)
43          BYTE ARRAY bytes[4] <- data TO BYTE ARRAY
44
45          SendData(bytes)
46      ENDSUBROUTINE
47
48      PRIVATE SUBROUTINE SendData (BYTE ARRAY bytes)
49          WRITE Serialise(bytes) TO _cryptoStream
50          FLUSH _cryptoStream
51      ENDSUBROUTINE
52
53      PUBLIC SUBROUTINE Finish()
54          FLUSH FINAL BLOCK OF _cryptoStream
55      ENDSUBROUTINE
56  ENDCLASS
57
58  CLASS FileReceiver FROM Transferer
59      PUBLIC CONSTRUCTOR FileReceiver (NETWORK STREAM stream, AES
60          aes)
61          CALL BASE(stream, aes, READ)
62      ENDCONSTRUCTOR

```

```

61     PUBLIC SUBROUTINE GetFile (STRING location, BOOLEAN decompress)
62         BYTE ARRAY hashBytes <- ReadByteArray(_cryptoStream)
63         BYTE ARRAY nameBytes <- ReadByteArray(_cryptoStream)
64         BYTE ARRAY fileBytes <- ReadByteArray(_cryptoStream)
65
66         IF (VerifyHash(fileBytes, hashBytes) = TRUE) THEN
67             STRING name <- nameBytes TO STRING
68             STRING filePath <- location + name
69
70             CREATE DIRECTORY AT filePath
71
72             IF (decompress = TRUE AND LAST ELEMENT OF
73                 (SPLIT name BY '.' ) = "huff") THEN
74                 STRING outputPath <-
75                     SUBSTRING(filePath, 0, LENGTH OF
76                         filePath - 5)
77
78                 HuffmanDecoder.WriteRawFile(fileBytes,
79                     outputPath)
80             ELSE
81                 WRITE fileBytes TO FILE filePath
82             ENDIF
83         ELSE
84             OUTPUT "Corrupted transmission!"
85         ENDIF
86     ENDSUBROUTINE
87
88     PUBLIC SUBROUTINE GetString()
89         BYTE ARRAY hashBytes <- ReadByteArray(_cryptoStream)
90         BYTE ARRAY stringBytes <- ReadByteArray(_cryptoStream)
91
92         IF (VerifyHash(stringBytes, hashBytes) = TRUE) THEN
93             RETURN stringBytes TO STRING
94         ENDIF
95
96         RETURN ""
97     ENDSUBROUTINE
98
99     PUBLIC SUBROUTINE GetInt()
100         BYTE ARRAY intBytes <- ReadByteArray(_cryptoStream)
101
102         RETURN intBytes TO INTEGER
103     ENDSUBROUTINE
104 ENDCLASS

```

Listing 18: Pseudocode for the transfer classes

```

1  using System.Net.Sockets;
2  using System.Security.Cryptography;
3
4  abstract class Transferer

```



```

5 {
6     protected CryptoStream _cryptoStream;
7
8     public Transferer(NetworkStream stream, Aes aes, CryptoStreamMode
        mode)
9     {
10         aes.Padding = PaddingMode.Zeros;
11
12         _cryptoStream = new CryptoStream(stream, mode ==
            CryptoStreamMode.Write ? aes.CreateEncryptor() :
            aes.CreateDecryptor(), mode, leaveOpen: true);
13     }
14 }
15
16 class FileSender : Transferer
17 {
18     public FileSender(NetworkStream stream, Aes aes) : base(stream,
        aes, CryptoStreamMode.Write) { }
19
20     public void SendFile(byte[] fileBytes, string fileName)
21     {
22         _cryptoStream.Write(
23             Transmission.Serialise(Hashing.Hash(fileBytes))
24         );
25
26         _cryptoStream.Write(Transmission.Serialise(fileName));
27
28         SendData(fileBytes);
29     }
30
31     public void SendData(string data)
32     {
33         byte[] bytes = System.Text.Encoding.ASCII.GetBytes(data);
34
35         _cryptoStream.Write(
36             Transmission.Serialise(Hashing.Hash(bytes))
37         );
38
39         SendData(bytes);
40     }
41
42     public void SendInt(int data)
43     {
44         byte[] bytes = BitConverter.GetBytes(data); // 4 bytes
45
46         SendData(bytes);
47     }
48
49     private void SendData(byte[] bytes)
50     {
51         _cryptoStream.Write(Transmission.Serialise(bytes));

```

```

52     _cryptoStream.Flush();
53 }
54
55     public void Finish() => _cryptoStream.FlushFinalBlock();
56 }
57
58 class FileReceiver : Transferer
59 {
60     public FileReceiver(NetworkStream stream, Aes aes) : base(stream,
61         aes, CryptoStreamMode.Read) { }
62
63     public void GetFile(string location, bool decompress = false)
64     {
65         byte[] hashBytes = Transmission.ReadByteArray(_cryptoStream);
66         // 32 byte hash read
67         byte[] nameBytes = Transmission.ReadByteArray(_cryptoStream);
68         // read file name
69         byte[] fileBytes = Transmission.ReadByteArray(_cryptoStream);
70
71         if (Hashing.VerifyHash(fileBytes, hashBytes))
72         {
73             string name =
74                 System.Text.Encoding.ASCII.GetString(nameBytes);
75
76             string filePath = location + name;
77
78             Directory.CreateDirectory(Path.GetDirectoryName(filePath)
79                 ?? "");
80
81             if (decompress && name.Split('.').LastOrDefault() ==
82                 "huff")
83             {
84                 HuffmanDecoder.WriteRawFile(fileBytes,
85                     filePath.Substring(0, filePath.Length - 5));
86             }
87             else File.WriteAllBytes(filePath, fileBytes);
88         }
89         else Console.WriteLine("Corrupted transmission!");
90     }
91
92     public string GetString()
93     {
94         byte[] hashBytes = Transmission.ReadByteArray(_cryptoStream);
95         byte[] stringBytes = Transmission.ReadByteArray(_cryptoStream);
96
97         if (Hashing.VerifyHash(stringBytes, hashBytes))
98         {
99             return System.Text.Encoding.ASCII.GetString(stringBytes);
100         }
101
102         return "";

```

```

96     }
97
98     public int GetInt()
99     {
100         byte[] intBytes = Transmission.ReadByteArray(_cryptoStream);
101
102         return BitConverter.ToInt32(intBytes);
103     }
104 }

```

Listing 19: Code for the transfer classes

2.4.1 UML Class Diagram

A UML class diagram for the above classes can be seen in Figure 13, demonstrating the inheritance from the base Transferer class.

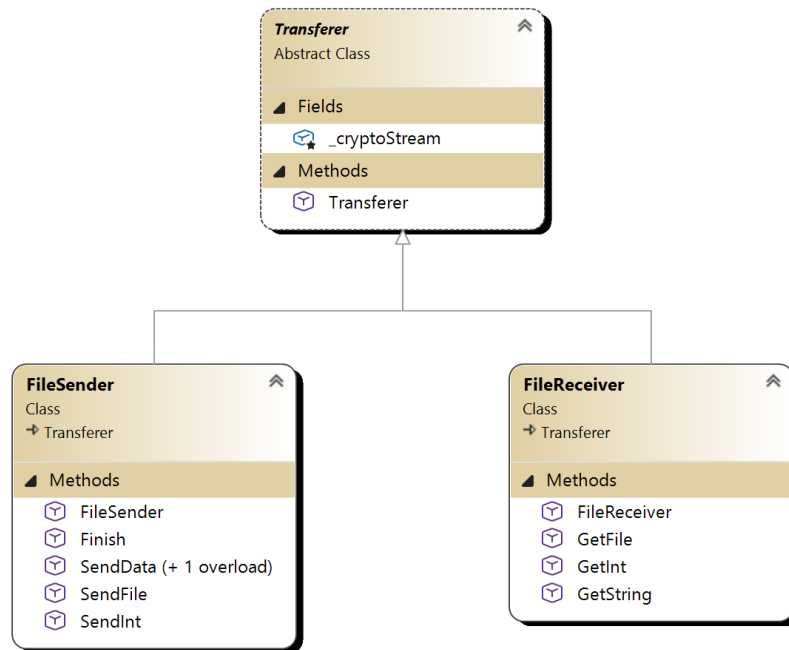


Figure 13: A UML class diagram for the Transferer, FileSender, and FileReceiver classes

2.5 Full UML Class Diagram

Figure 14 shows a UML class diagram for my design of the whole program. Polymorphism is often shown through overloaded methods, and there are two derived classes. However, often, classes are static and simply used for the grouping of related methods.

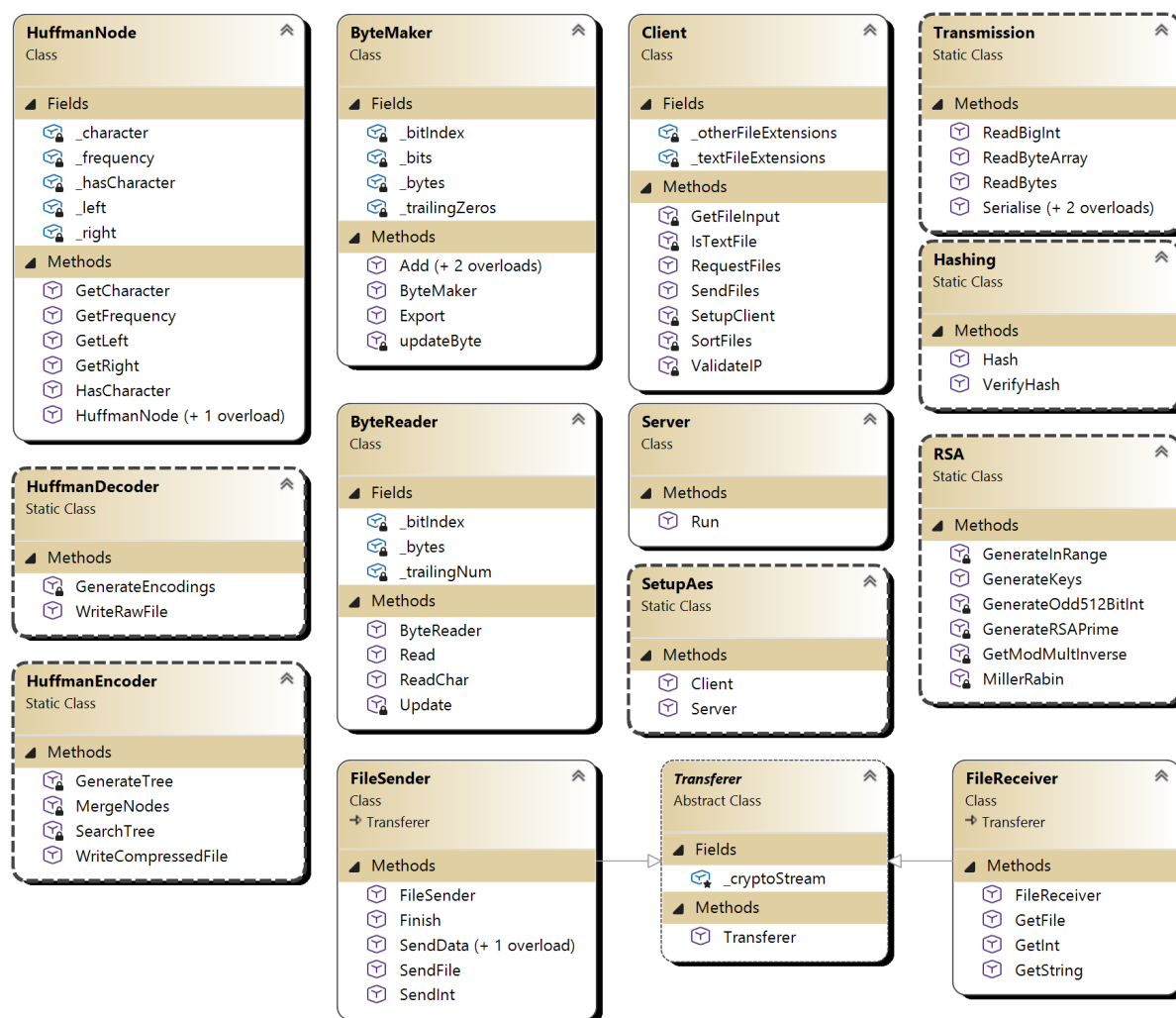


Figure 14: A UML class diagram for the whole program

3 Technical Solution

3.1 Main Program

```
1 Client client;
2 Server server;
3
4 if (args.Length == 0)
5 {
6     PrintFormatDetails();
7     return;
8 }
9
10 switch (args[0])
11 {
12     case "-cs":
13         client = new Client();
14         client.SendFiles(args);
15
16         break;
17
18     case "-cr":
19         client = new Client();
20         client.RequestFiles(args);
21
22         break;
23
24     case "-s":
25         server = new Server();
26         server.Run(args);
27
28         break;
29
30     default:
31         PrintFormatDetails();
32
33         break;
34 }
35
36 void PrintFormatDetails()
37 {
38     Console.WriteLine("Format:");
39     Console.WriteLine();
40     Console.WriteLine("-cs (Client Send)      [Server IP] [Server
        Port] [File1] t/o (t for text file, o for other file)
        [File2] ...");
41     Console.WriteLine("-cr (Client Receive) [Server IP] [Server
        Port] [File1] t/o (t for text file, o for other file)
        [File2] ...");
```

```

42     Console.WriteLine("-s (Server) [Port] loop (enter if the
        server should be run on the loopback address)");
43     Console.WriteLine();
44     Console.WriteLine("Any details not provided will be requested
        interactively, but the initial flag is mandatory.");
45     Console.WriteLine();
46     Console.WriteLine("Note that providing any details via CLI
        flags depends on the previous details also being entered:
        you cannot pass files as an argument without entering the
        server IP and port!");
47 }

```

Listing 20: Program.cs

```

1  using System.Net.Sockets;
2  using System.Security.Cryptography;
3
4  class Client
5  {
6      private static readonly IReadOnlyList<string> _textFileExtensions
          = new string[]
7      {
8          "123", "1st", "600", "602", "890", "a", "ab2", "ab3", "abc", "abw",
          "yaml", "ynab", "yum", "zabw", "zeg", "zig", "zrtf", "zsh", "zw"
9      }; // large list of plaintext file extensions - most have been
          removed for this writeup for clarity
10     private static readonly IReadOnlyList<string> _otherFileExtensions
          = new string[]
11     {
12         "exe", "dll", "so", "bin", "msi", "app", "dmg", "zip", "rar", "7z",
          "gz", "tar", "tgz", "bz2", "xz", "iso", "jpg", "jpeg", "png", "gif"
13     }; // large list of other file extensions - most have been removed
          for this writeup for clarity
14
15     public void SendFiles(string[] args)
16     {
17         TcpClient client = SetupClient(args);
18         NetworkStream stream = client.GetStream();
19
20         List<string> textFiles = new List<string>();
21         List<string> otherFiles = new List<string>();
22
23         if (args.Length > 3)
24         {
25             SortFiles(textFiles, otherFiles, args);
26         }
27         else
28         {
29             Console.WriteLine("Enter file paths to send (blank line to
                finish)");
30             GetFileInput(textFiles, otherFiles);

```

```
31     }
32
33     using (client)
34     using (stream)
35     {
36         Aes aes = SetupAes.Client(stream);
37
38         FileSender sender = new FileSender(stream, aes);
39
40         List<Tuple<byte[], string>> filesToSend = new
41             List<Tuple<byte[], string>>();
42
43         foreach (string path in textFiles)
44         {
45             string fileText = "";
46
47             try
48             {
49                 fileText = File.ReadAllText(path);
50             }
51             catch
52             {
53                 Console.WriteLine(path + " is invalid, so not
54                     sent!");
55                 continue;
56             }
57
58             filesToSend.Add(new Tuple<byte[], string>
59                 (HuffmanEncoder.WriteCompressedFile(fileText),
60                     path + ".huff"));
61         }
62
63         foreach (string path in otherFiles)
64         {
65             byte[] fileBytes;
66
67             try
68             {
69                 fileBytes = File.ReadAllBytes(path);
70             }
71             catch
72             {
73                 Console.WriteLine(path + " is invalid, so not
74                     sent!");
75                 continue;
76             }
77
78             filesToSend.Add(new Tuple<byte[], string>(fileBytes,
79                 path));
80         }
81     }
```

```

77         sender.SendInt(filesToSend.Count); // no. of files to
           expect
78
79         foreach (Tuple<byte[], string> file in filesToSend) //
           sent after so non-existent files don't contribute to
           count
80         {
81             Console.WriteLine("Sending " + file.Item2);
82             sender.SendFile(file.Item1, file.Item2);
83         }
84
85         sender.Finish();
86     }
87 }
88
89 public void RequestFiles(string[] args)
90 {
91     TcpClient client = SetupClient(args);
92     NetworkStream stream = client.GetStream();
93
94     using (client)
95     using (stream)
96     {
97         Aes aes = SetupAes.Client(stream);
98         FileSender sender = new FileSender(stream, aes);
99
100        sender.SendInt(0); // indicates desire to receive files
101
102        List<string> textFiles = new List<string>();
103        List<string> otherFiles = new List<string>();
104
105        if (args.Length > 3)
106        {
107            SortFiles(textFiles, otherFiles, args);
108        }
109        else
110        {
111            Console.WriteLine("Enter file paths to receive (blank
              line to finish)");
112            GetFileInput(textFiles, otherFiles);
113        }
114
115        string paths = "";
116
117        foreach (string path in textFiles) paths += path + ".huff"
            + ",";
118        foreach (string path in otherFiles) paths += path + ",";
119
120        paths = paths.Remove(paths.Length - 1);
121
122        sender.SendData(paths);

```



```
123         sender.Finish();
124
125         FileReceiver receiver = new FileReceiver(stream, aes);
126
127         for (int i = 0; i < textFiles.Count + otherFiles.Count;
128             i++)
129         {
130             receiver.GetFile("ClientReceived", true);
131         }
132     }
133 }
134
135 private TcpClient SetupClient(string[] args)
136 {
137     string serverIp = "";
138     int serverPort;
139
140     if (args.Length > 1)
141     {
142         serverIp = args[1];
143     }
144     if (args.Length <= 1 || !ValidateIP(serverIp)) // write this
145         please
146     {
147         do
148         {
149             Console.Write("Enter the server's IP address: ");
150             serverIp = Console.ReadLine() ?? "127.0.0.1";
151         } while (!ValidateIP(serverIp));
152     }
153     if (!(args.Length > 2 && int.TryParse(args[2], out
154         serverPort)))
155     {
156         Console.Write("Enter the server's port: ");
157
158         while (!int.TryParse(Console.ReadLine(), out serverPort))
159         {
160             Console.WriteLine("Invalid port! It must be an
161                 integer! Try again.");
162         }
163     }
164     try
165     {
166         return new TcpClient(serverIp, serverPort);
167     }
168     catch
169     {
170         Console.WriteLine("Error! Server not reached!");
171     }
172 }
```

```

170         return SetupClient(new string[]{}); // forces restart in
           interactive mode
171     }
172 }
173
174 private bool ValidateIP(string ipAddress)
175 {
176     if (!ipAddress.Contains('.')) return false;
177
178     string[] splitIp = ipAddress.Split('.');
179
180     if (splitIp.Length != 4) return false;
181
182     foreach (string str in splitIp)
183     {
184         int ipPart;
185
186         if (!int.TryParse(str, out ipPart)) return false;
187
188         if (ipPart > 255 || ipPart < 0) return false;
189     }
190
191     return true;
192 }
193
194 private void GetFileInput(List<string> textFiles, List<string>
           otherFiles) // interactive
195 {
196     while (true)
197     {
198         string input = Console.ReadLine() ?? "";
199
200         if (input == "") break;
201
202         if (IsTextFile(input, true)) textFiles.Add(input);
203         else otherFiles.Add(input);
204
205     }
206 }
207
208 private void SortFiles(List<string> textFiles, List<string>
           otherFiles, string[] args) // non-interactive
209 {
210     for (int i = 3; i < args.Length; i++)
211     {
212         string fileName = args[i];
213         string flag = (i + 1 < args.Length) ? args[i+1].ToLower()
           : "";
214
215         if (flag == "t")
216         {

```

```

217         textFiles.Add(fileName);
218         i++;
219         continue;
220     }
221     if (flag == "o")
222     {
223         otherFiles.Add(fileName);
224         i++;
225         continue;
226     }
227
228     if (IsTextFile(fileName, false)) textFiles.Add(fileName);
229     else otherFiles.Add(fileName);
230 }
231 }
232
233 private bool IsTextFile(string fileName, bool interactive)
234 {
235     string extension;
236
237     if (!fileName.Contains('.')) extension = "";
238     else extension = fileName.Split('.').Last();
239
240     if (_textFileExtensions.Contains(extension)) return true;
241     if (_otherFileExtensions.Contains(extension)) return false;
242
243     if (!interactive) return false;
244
245     Console.Write("Plaintext? (y/N) ");
246     if ((Console.ReadLine() ?? "").ToLower() == "y") return true;
247
248     return false;
249 }
250 }

```

Listing 21: Client.cs

```

1  using System.Net;
2  using System.Net.Sockets;
3  using System.Security.Cryptography;
4
5  class Server
6  {
7      public void Run(string[] args)
8      {
9          int port;
10
11          if (args.Length < 2 || !int.TryParse(args[1], out port))
12          {
13              Console.WriteLine("Provide an integer port as the second
              argument!");

```

```
14         return;
15     }
16
17     IPAddress ip;
18     if (args.Length > 2 && args[2] == "loop") ip =
19         IPAddress.Loopback;
20     else ip = IPAddress.Any;
21
22     TcpListener listener = new TcpListener(ip, port);
23     listener.Start();
24
25     Console.WriteLine("IP: " + ip + " Port: " + port);
26
27     Aes aes;
28
29     while (true)
30     {
31         using (NetworkStream stream =
32             listener.AcceptTcpClient().GetStream())
33         {
34             aes = SetupAes.Server(stream);
35
36             FileReceiver receiver = new FileReceiver(stream, aes);
37
38             int fileNum = receiver.GetInt();
39
40             for (int i = 0; i < fileNum; i++) // client sending
41                 files
42             {
43                 receiver.GetFile("ServerReceived");
44             }
45
46             if (fileNum == 0) // client retrieving files
47             {
48                 string paths = receiver.GetString();
49
50                 if (paths == "")
51                 {
52                     Console.WriteLine("Corrupted transmission!");
53                     continue;
54                 }
55
56                 FileSender sender = new FileSender(stream, aes);
57
58                 foreach (string path in paths.Trim().Split(','))
59                 {
50                     try
51                     {
52                         byte[] fileBytes =
53                             File.ReadAllBytes("ServerReceived" +
54                                 path);
55                     }
56                     catch { }
57                 }
58             }
59         }
60     }
```

```

60
61         sender.SendFile(fileBytes, path);
62     }
63     catch
64     {
65         sender.SendFile(
66             System.Text.Encoding.ASCII.GetBytes
67             ("File not found!"), path);
68
69         Console.WriteLine(path + " not found!");
70     }
71 }
72
73     sender.Finish();
74 }
75 }
76 }
77 }
78 }

```

Listing 22: Server.cs

3.2 Huffman Compression

```

1  class ByteMaker
2  {
3      private int[] _bits;
4      private int _bitIndex;
5      private int _trailingZeros;
6
7      private List<byte> _bytes;
8
9      public ByteMaker(bool isCompressed)
10     {
11         _bits = [isCompressed ? 1 : 0, 0, 0, 0, 0, 0, 0, 0]; // length
12                     8
13         _bitIndex = 4; // first bit indicates compressed, next 3 bits
14                         reserved to give data about trailing zeros
15         _trailingZeros = 0;
16
17         _bytes = new List<byte>();
18     }
19
20     public void Add(int bit)
21     {
22         if (bit != 1 && bit != 0) return;
23
24         _bits[_bitIndex++] = bit;
25
26         if (_bitIndex == 8) updateByte();
27     }
28 }

```

```

25     }
26
27     public void Add(List<int> bits)
28     {
29         foreach (int bit in bits) Add(bit);
30     }
31
32     public void Add(char character)
33     {
34         byte characterByte = (byte)character;
35
36         for (int i = 7; i >= 0; i--)
37         {
38             int bit = (characterByte >> i) & 1; // right-shift, then
39                 AND with 1 to get "last" bit
40
41             Add(bit);
42         }
43
44     public byte[] Export()
45     {
46         updateByte();
47
48         _bytes[0] |= (byte)(_trailingZeros << 4); // push to bits 1-4,
49             OR since already 0
50
51         return _bytes.ToArray();
52     }
53
54     private void updateByte()
55     {
56         byte newByte = new byte();
57
58         for (int i = 0; i < 8; i++)
59         {
60             newByte |= (byte)(_bits[i] << (7-i)); // left-shift, then
61                 OR with existing byte
62         }
63
64         _bytes.Add(newByte);
65
66         _bits = [0, 0, 0, 0, 0, 0, 0, 0]; // length 8
67         _trailingZeros = 8 - _bitIndex;
68         _bitIndex = 0;
69     }
70 }

```

Listing 23: Huffman/ByteMaker.cs

```

1 class ByteReader

```

```

2 {
3     private List<byte> _bytes;
4     private int _bitIndex;
5     private int _trailingNum;
6
7     public ByteReader(byte[] bytes, out bool compressed)
8     {
9         _bytes = bytes.ToList();
10        _bitIndex = 0;
11
12        int firstBit = Read();
13
14        if (firstBit == 0) compressed = false;
15        else compressed = true;
16
17        int a = Read();
18        int b = Read();
19        int c = Read();
20
21        _trailingNum = a*4 + b*2 + c;
22    }
23
24    public int Read()
25    {
26        if (_bytes.Count() == 0) return -1;
27
28        if (_bytes.Count() == 1 && _bitIndex >= 8 - _trailingNum)
29            return -1; // don't read padding zeros
30
31        int bit = (_bytes[0] >> (7 - _bitIndex++)) & 1;
32
33        Update();
34
35        return bit;
36    }
37
38    public char? ReadChar()
39    {
40        byte nextByte = 0;
41
42        for (int i = 0; i < 8; i++)
43        {
44            int bit = Read();
45            if (bit == -1) return null;
46
47            if (bit == 1) nextByte |= (byte)(1 << (7-i)); // shift 1
48                to its position and OR with existing 00...0 byte
49        }
50
51        return (char)nextByte;
52    }

```

```

51
52     private void Update()
53     {
54         if (_bitIndex != 8) return;
55
56         _bytes.RemoveAt(0);
57         _bitIndex = 0;
58     }
59 }

```

Listing 24: Huffman/ByteReader.cs

```

1  static class HuffmanEncoder
2  {
3      public static byte[] WriteCompressedFile(string text)
4      {
5          HuffmanNode tree = GenerateTree(text);
6
7          // add encoding of tree
8          ByteMaker byteMaker = new ByteMaker(true);
9
10         Dictionary<char, List<int>> CharacterEncodings = new
11             Dictionary<char, List<int>>();
12
13         SearchTree(tree, byteMaker, CharacterEncodings, new
14             List<int>());
15         byteMaker.Add(0);
16
17         // add encoding of text
18         foreach (char c in text) byteMaker.Add(CharacterEncodings[c]);
19
20         byte[] bytes = byteMaker.Export();
21
22         if (bytes.Length > text.Length)
23         {
24             ByteMaker textByteMaker = new ByteMaker(false);
25
26             foreach(char c in text) textByteMaker.Add(c);
27
28             bytes = textByteMaker.Export();
29         }
30
31         return bytes;
32     }
33
34     private static HuffmanNode GenerateTree(string text)
35     {
36         // link characters to their count
37         Dictionary<char, int> CharacterCounts = new Dictionary<char,
38             int>();
39     }

```



```

37     foreach (char c in text)
38     {
39         if (CharacterCounts.ContainsKey(c)) CharacterCounts[c]++;
40         else CharacterCounts[c] = 1;
41     }
42
43     // place characters in priorityqueue (based on freq) to
44     // generate the tree
45     PriorityQueue<HuffmanNode, int> NodeQueue = new
46         PriorityQueue<HuffmanNode, int>(); // "base" trees of size 1
47
48     foreach (KeyValuePair<char, int> kvp in CharacterCounts)
49     {
50         NodeQueue.Enqueue(new HuffmanNode(kvp.Key, kvp.Value),
51             kvp.Value);
52     }
53
54     HuffmanNode tree = MergeNodes(NodeQueue); // tree = top node
55     // of tree
56
57     return tree;
58 }
59
60 // recursively merge nodes to make a tree
61 private static HuffmanNode MergeNodes(PriorityQueue<HuffmanNode,
62     int> trees)
63 {
64     if (trees.Count == 1) return trees.Dequeue();
65
66     HuffmanNode tree1 = trees.Dequeue();
67     HuffmanNode tree2 = trees.Dequeue();
68
69     int newFreq = tree1.GetFrequency() + tree2.GetFrequency();
70
71     HuffmanNode newTree = new HuffmanNode(tree1, tree2, newFreq);
72
73     trees.Enqueue(newTree, newFreq);
74
75     return MergeNodes(trees);
76 }
77
78 // fill byteMaker with encoded tree, and CharacterEncodings with
79 // bits encoding characters, uses post order DFS
80 private static void SearchTree(HuffmanNode tree, ByteMaker
81     byteMaker, Dictionary<char, List<int>> CharacterEncodings,
82     List<int> current)
83 {
84     if (tree == null) return;
85
86     SearchTree(tree.GetLeft(), byteMaker, CharacterEncodings,
87         current.Append(0).ToList());

```

```

79     SearchTree(tree.GetRight(), byteMaker, CharacterEncodings,
80               current.Append(1).ToList());
81
82     if (tree.HasCharacter()) // is leaf
83     {
84         byteMaker.Add(1);
85         byteMaker.Add(tree.GetCharacter());
86
87         CharacterEncodings[tree.GetCharacter()] = current;
88     }
89     else
90     {
91         byteMaker.Add(0);
92     }
93 }

```

Listing 25: Huffman/HuffmanEncoder.cs

```

1  static class HuffmanDecoder
2  {
3      public static void WriteRawFile(byte[] bytes, string path)
4      {
5          bool fileCompressed;
6
7          ByteReader byteReader = new ByteReader(bytes, out
8              fileCompressed);
9          string rawFile = "";
10
11         if (!fileCompressed)
12         {
13             while (true)
14             {
15                 char? nextChar = byteReader.ReadChar();
16
17                 if (nextChar == null) break;
18
19                 rawFile += nextChar;
20             }
21         }
22         else
23         {
24             Stack<HuffmanNode> NodeStack = new Stack<HuffmanNode>();
25
26             while (true)
27             {
28                 int nextBit = byteReader.Read();
29
30                 if (nextBit == 1)
31                 {
32                     char? nextChar = byteReader.ReadChar();

```

```

32         if (nextChar == null) break;
33
34         NodeStack.Push(new HuffmanNode((char)nextChar));
35     }
36     else
37     {
38         if (NodeStack.Count() == 1) break;
39
40         HuffmanNode right = NodeStack.Pop();
41         HuffmanNode left = NodeStack.Pop();
42
43         NodeStack.Push(new HuffmanNode(left, right));
44     }
45 }
46
47 HuffmanNode tree = NodeStack.Pop(); // there should be
    just one node left
48
49 Dictionary<string, char> EncodingCharacters = new
    Dictionary<string, char>(); // maps encodings to
    character
50 GenerateEncodings(tree, EncodingCharacters, "");
51
52 string currentEncoding = ""; // string, not int array, as
    arrays/lists compared by instance, not contents
53
54 while (true)
55 {
56     int nextBit = byteReader.Read();
57     if (nextBit == -1) break;
58
59     currentEncoding += nextBit.ToString();
60
61     if (!EncodingCharacters.ContainsKey(currentEncoding))
        continue;
62
63     rawFile += EncodingCharacters[currentEncoding];
64
65     currentEncoding = "";
66 }
67
68
69 try
70 {
71     File.WriteAllText(path, rawFile);
72 }
73 catch
74 {
75     Console.WriteLine("Error: Could not write to " + path);
76 }
77

```

```

78
79     private static void GenerateEncodings(HuffmanNode tree,
80         Dictionary<string, char> EncodingCharacters, string current)
81     {
82         if (tree == null) return;
83
84         GenerateEncodings(tree.GetLeft(), EncodingCharacters, current
85             + "0");
86         GenerateEncodings(tree.GetRight(), EncodingCharacters, current
87             + "1");
88
89         if (tree.HasCharacter()) EncodingCharacters[current] =
90             tree.GetCharacter();
91     }
92 }

```

Listing 26: Huffman/HuffmanDecoder.cs

```

1  class HuffmanNode
2  {
3      private char _character;
4      private int _frequency;
5      private HuffmanNode? _left;
6      private HuffmanNode? _right;
7
8      private bool _hasCharacter; // "isLeaf"
9
10     public HuffmanNode(char character, int frequency = 1)
11     {
12         _character = character;
13         _frequency = frequency;
14
15         _hasCharacter = true;
16     }
17
18     public HuffmanNode(HuffmanNode left, HuffmanNode right, int
19         frequency = 1)
20     {
21         _left = left;
22         _right = right;
23         _frequency = frequency;
24
25         _hasCharacter = false;
26     }
27
28     public int GetFrequency() => _frequency;
29
30     public HuffmanNode? GetLeft() => _left;
31     public HuffmanNode? GetRight() => _right;
32
33     public bool HasCharacter() => _hasCharacter;

```

```

33     public char GetCharacter() => _character;
34 }
35
36 // NOTE: tree is just a "top-node" - links to left/right

```

Listing 27: Huffman/HuffmanNode.cs

3.3 Transmission Procedure

```

1  using System.Numerics;
2  using System.Net.Sockets;
3
4  static class Transmission
5  {
6      public static byte[] Serialise(byte[] bytes)
7      {
8          byte[] bytesLengthBytes = BitConverter.GetBytes(bytes.Length);
9          // length 4 always
10
11         byte[] data =
12             bytesLengthBytes.ToList().Concat(bytes.ToList()).ToArray();
13
14         return data;
15     }
16
17     public static byte[] Serialise(BigInteger bigInt)
18     {
19         byte[] bigIntBytes = bigInt.ToByteArray(true, true); //
20         // isUnsigned, isBigEndian
21
22         return Serialise(bigIntBytes);
23     }
24
25     public static byte[] Serialise(string data) =>
26         Serialise(System.Text.Encoding.ASCII.GetBytes(data));
27
28     public static BigInteger ReadBigInt(NetworkStream stream) => new
29         BigInteger(ReadByteArray(stream), true, true);
30
31     public static byte[] ReadByteArray(Stream stream)
32     {
33         int bytesLength = BitConverter.ToInt32(ReadBytes(stream, 4));
34
35         return ReadBytes(stream, bytesLength);
36     }
37
38     public static byte[] ReadBytes(Stream stream, int length)
39     {
40         byte[] bytes = new byte[length];

```

```

37     int read = 0;
38     while (read < length) read += stream.Read(bytes, read, length
39         - read);
40     return bytes;
41 }
42 }

```

Listing 28: Transmission/Transmission.cs

```

1  using System.Net.Sockets;
2  using System.Security.Cryptography;
3
4  abstract class Transferer
5  {
6      protected CryptoStream _cryptoStream;
7
8      public Transferer(NetworkStream stream, Aes aes, CryptoStreamMode
9          mode)
10     {
11         aes.Padding = PaddingMode.Zeros;
12
13         _cryptoStream = new CryptoStream(stream, mode ==
14             CryptoStreamMode.Write ? aes.CreateEncryptor() :
15             aes.CreateDecryptor(), mode, leaveOpen: true);
16     }
17 }
18
19 class FileSender : Transferer
20 {
21     public FileSender(NetworkStream stream, Aes aes) : base(stream,
22         aes, CryptoStreamMode.Write) { }
23
24     public void SendFile(byte[] fileBytes, string fileName)
25     {
26         _cryptoStream.Write(
27             Transmission.Serialise(Hashing.Hash(fileBytes))
28         );
29
30         _cryptoStream.Write(Transmission.Serialise(fileName));
31
32         SendData(fileBytes);
33     }
34
35     public void SendData(string data)
36     {
37         byte[] bytes = System.Text.Encoding.ASCII.GetBytes(data);
38
39         _cryptoStream.Write(
40             Transmission.Serialise(Hashing.Hash(bytes))
41         );

```

```

38         SendData(bytes);
39     }
40
41
42     public void SendInt(int data) // different name since no hashing
43     {
44         byte[] bytes = BitConverter.GetBytes(data); // 4 bytes
45
46         SendData(bytes);
47     }
48
49     private void SendData(byte[] bytes)
50     {
51         _cryptoStream.Write(Transmission.Serialise(bytes));
52         _cryptoStream.Flush();
53     }
54
55     public void Finish() => _cryptoStream.FlushFinalBlock();
56 }
57
58 class FileReceiver : Transferer
59 {
60     public FileReceiver(NetworkStream stream, Aes aes) : base(stream,
61         aes, CryptoStreamMode.Read) { }
62
63     public void GetFile(string location, bool decompress = false)
64     {
65         byte[] hashBytes = Transmission.ReadByteArray(_cryptoStream);
66         // 32 byte hash read
67         byte[] nameBytes = Transmission.ReadByteArray(_cryptoStream);
68         // read file name
69         byte[] fileBytes = Transmission.ReadByteArray(_cryptoStream);
70
71         if (Hashing.VerifyHash(fileBytes, hashBytes))
72         {
73             string name =
74                 System.Text.Encoding.ASCII.GetString(nameBytes);
75
76             string filePath = location + name;
77
78             Directory.CreateDirectory(Path.GetDirectoryName(filePath)
79                 ?? "");
80
81             if (decompress && name.Split('.').LastOrDefault() ==
82                 "huff")
83             {
84                 HuffmanDecoder.WriteRawFile(fileBytes,
85                     filePath.Substring(0, filePath.Length - 5));
86             }
87             else File.WriteAllBytes(filePath, fileBytes);
88         }
89     }
90 }

```

```

82         else Console.WriteLine("Corrupted transmission!");
83     }
84
85     public string GetString()
86     {
87         byte[] hashBytes = Transmission.ReadByteArray(_cryptoStream);
88         byte[] stringBytes = Transmission.ReadByteArray(_cryptoStream);
89
90         if (Hashing.VerifyHash(stringBytes, hashBytes))
91         {
92             return System.Text.Encoding.ASCII.GetString(stringBytes);
93         }
94
95         return "";
96     }
97
98     public int GetInt()
99     {
100         byte[] intBytes = Transmission.ReadByteArray(_cryptoStream);
101
102         return BitConverter.ToInt32(intBytes);
103     }
104 }

```

Listing 29: Transmission/Transferer.cs

```

1  using System.Security.Cryptography;
2  using System.Numerics;
3
4  static class RSA
5  {
6      public static Tuple<BigInteger, BigInteger, BigInteger>
7          GenerateKeys()
8      {
9          BigInteger e = 65537;
10
11          BigInteger p = RSA.GenerateRSAPrime(e);
12          BigInteger q = RSA.GenerateRSAPrime(e);
13
14          BigInteger n = p*q;
15          BigInteger phin = (p-1)*(q-1);
16
17          BigInteger d = RSA.GetModMultInverse(e, phin);
18
19          return new Tuple<BigInteger, BigInteger, BigInteger>(n,e,d);
20      }
21
22      private static BigInteger GenerateRSAPrime(BigInteger e) // e must
23          be prime in this implementation!
24      {
25          while (true)

```



```

24     {
25         BigInteger bigInt;
26
27         do bigInt = GenerateOdd512BitInt();
28         while (bigInt % e == 1); // ensures e will be relatively
           prime with phi(n), as e used is prime
29
30         if (MillerRabin(bigInt, 40)) return bigInt;
31     }
32 }
33
34 private static BigInteger GetModMultInverse(BigInteger a,
           BigInteger m) // uses the extended euclidean algorithm to find
           x s.t. ax = 1 (mod m)
35 {
36     BigInteger r_0 = a;
37     BigInteger s_0 = 1;
38     BigInteger t_0 = 0;
39
40     BigInteger r_1 = m;
41     BigInteger s_1 = 0;
42     BigInteger t_1 = 1;
43
44     BigInteger r_2, s_2, t_2;
45
46     do
47     {
48         BigInteger q = r_0 / r_1;
49
50         r_2 = r_0 - q * r_1;
51         s_2 = s_0 - q * s_1;
52         t_2 = t_0 - q * t_1;
53
54         (r_0, r_1) = (r_1, r_2);
55         (s_0, s_1) = (s_1, s_2);
56         (t_0, t_1) = (t_1, t_2);
57
58     } while (r_2 != 0);
59
60     BigInteger d = s_0 % m;
61     if (d < 0) d += m;
62
63     return d;
64 }
65
66 private static BigInteger GenerateOdd512BitInt()
67 {
68     byte[] randomInt = new byte[64]; // 512 bits = 64 bytes
69     RandomNumberGenerator.Fill(randomInt);
70
71     randomInt[0] |= (byte)(128); // sets MSB to 1, so num is 512

```

```

        bits
72
73         randomInt[63] |= (byte)(1); // sets LSB to 1 - odd
74
75         return new BigInteger(randomInt, true, true); // isUnsigned,
           isBigEndian
76     }
77
78     private static BigInteger GenerateInRange(BigInteger lower,
           BigInteger upper)
79     {
80         BigInteger num = -1;
81
82         while (num < lower || num > upper)
83         {
84             byte[] randomInt = new byte[64]; // 512 bits = 64 bytes
85             RandomNumberGenerator.Fill(randomInt);
86
87             num = new BigInteger(randomInt, true, true);
88         }
89
90         return num;
91     }
92
93     private static bool MillerRabin(BigInteger n, int rounds)
94     {
95         int s = 1;
96         while ((n-1) % (BigInteger)Math.Pow(2, s+1) == 0) s++;
97
98         BigInteger d = (n-1)/(BigInteger)(Math.Pow(2,s));
99
100        for (int i = 0; i < rounds; i++)
101        {
102            BigInteger a = GenerateInRange(2, n-3);
103
104            BigInteger x = BigInteger.ModPow(a, d, n);
105
106            for (int j = 0; j < s; j++)
107            {
108                BigInteger y = BigInteger.ModPow(x, 2, n);
109
110                if (y == 1 && x != 1 && x != n-1) return false;
111
112                x = y;
113            }
114
115            if (x != 1) return false;
116        }
117
118        return true;
119    }

```

120 }

Listing 30: Transmission/RSA.cs

```

1 using System.Net.Sockets;
2 using System.Numerics;
3 using System.Security.Cryptography;
4
5 static class SetupAes
6 {
7     public static Aes Client(NetworkStream stream)
8     {
9         BigInteger n = Transmission.ReadBigInt(stream);
10        BigInteger e = Transmission.ReadBigInt(stream);
11
12        Aes aes = Aes.Create(); // creates key and IV
13
14        BigInteger plaintext = new BigInteger(aes.Key, true, true);
15
16        BigInteger ciphertext = BigInteger.ModPow(plaintext, e, n);
17
18        stream.Write(Transmission.Serialise(ciphertext)); // sends RSA
19        // encrypted AES key
20
21        aes.IV = Transmission.ReadByteArray(stream);
22
23        return aes;
24    }
25
26    public static Aes Server(NetworkStream stream)
27    {
28        Tuple<BigInteger, BigInteger, BigInteger> keys =
29            RSA.GenerateKeys(); // n, e, d
30
31        stream.Write(Transmission.Serialise(keys.Item1)); // send n
32        stream.Write(Transmission.Serialise(keys.Item2)); // send e
33
34        BigInteger ciphertext = Transmission.ReadBigInt(stream); //
35        // receive RSA encrypted AES key
36
37        BigInteger plaintext = BigInteger.ModPow(ciphertext,
38            keys.Item3, keys.Item1); // M = C^d (mod n)
39
40        byte[] aesKey = plaintext.ToByteArray(true, true); //
41        // isUnsigned, isBigEndian
42
43        Aes aes = Aes.Create();
44        aes.Key = aesKey;
45
46        aes.GenerateIV();
47        stream.Write(Transmission.Serialise(aes.IV)); // send IV

```

```
43
44     return aes;
45 }
46 }
```

Listing 31: Transmission/SetupAes.cs

```
1 using System.Security.Cryptography;
2
3 static class Hashing
4 {
5     public static byte[] Hash(byte[] messageBytes)
6     {
7         byte[] hashBytes;
8
9         using (SHA256 HashMaker = SHA256.Create())
10        {
11            hashBytes = HashMaker.ComputeHash(messageBytes);
12        }
13
14        return hashBytes;
15    }
16
17    public static bool VerifyHash(byte[] messageBytes, byte[] hash)
18    {
19        byte[] originalHashed = Hash(messageBytes);
20
21        if (originalHashed.Length != hash.Length) return false;
22
23        for (int i = 0; i < hash.Length; i++)
24        {
25            if (originalHashed[i] != hash[i]) return false;
26        }
27
28        return true;
29    }
30 }
```

Listing 32: Transmission/Hashing.cs

4 Testing

4.1 Testing Strategy

To verify that my program fulfils all of the criteria specified in the Specification, I will carry out two main forms of formal testing.

Firstly, I will record a video of the client and server devices each running the program, showing them sending files between them and verifying the integrity of files being sent, monitoring the processor and memory usage of the programs, ensuring compression reduces file size, verifying that the program handles unexpected input gracefully, as well as some other details.

However, some parts of the program, namely the encryption, cannot be easily verified through normal usage, since if the files were being sent insecurely, the user would not notice and the program could appear to still work correctly, while being vulnerable to attacks behind the scenes. Therefore, I will separately test and verify the functioning of a sample of these algorithms. Since I am using standard C# implementations for Hashing and AES implementation, I will include here the testing for my Miller-Rabin Primality Test and my implementation of the Extended Euclidean Algorithm, both of which I have implemented myself, and form part of the broader RSA key exchange.

4.2 Typical Use Video

Attached in the submission is a video showing a typical use of the programs. Note that throughout, 'client' refers to the device making the backups, and 'server' to the device receiving and storing them. The structure of the video is as follows:

1. The original contents of the five files to be backed up are shown on the client device. The first, test1.txt, is a plaintext file containing generated lorem ipsum text. The second file, test2.txt, is another plaintext file, this time containing the novella Strange Case of Dr Jekyll and Mr Hyde by Robert Louis Stevenson. The third file, test3.cs, is some sample C# source code (in fact, it is part of the code used in the program being run). The fourth file, test4.png, is a screenshot of my desktop. The final file, test5.mp4, is a video of the earth spinning.
2. Throughout, the programs are run in split-screen with the *htop* utility, showing computer resource usage.
3. The executable binary is used to run the server program on port 9999.
4. The same executable binary is used to run the client program interactively, with me entering the server's IP address and port, and then the file path of test1.txt, when prompted, with the program sending it to the server.
5. I verify that the server has received a compressed version of this file, stored with a .huff extension to indicate this. The file size of the compressed version on the server is checked

using the command `du -sh [filename]`, and then compared to the same command run on the original file on the client device.

6. The server is started again on port 9999. This time the client sends the other four test files using command line arguments only.
7. On the server device, I verify that all of these have been received, and similarly to before compare their file sizes against the original ones on the client device. Note that these files should all be in the `ServerReceived` directory on the server.
8. Using command line arguments only, I run a command on the client device to retrieve all five files from the server. I then verify that they are all present in the newly created `ClientReceived` directory on the client, and then verify that each file is identical in content to the original ones that were sent.
9. Finally, I carry out various tests involved invalid, incorrect, and missing interactive and command line input to verify that the programs handle these correctly and robustly.

The table below contains details on individual conclusions that can be drawn from the test video, with details on how they were tested, and which specification points they show the fulfilment of.

	Spec	Test steps	Expected result	Actual result	Time	Pass?
1	1a/3b	Run the client and server programs interactively	They should form a connection	They form a connection - this is shown by the client not throwing an error	0:51 - 1:35	Pass
2	1b/3b	Send a file from the client to the server interactively	The file should be accessible on the server	The file is accessible on the server (though in compressed form)	0:51 - 1:50	Pass
3	2a	Run the executable binary of the client and server programs	They should work as expected	They run as expected	0:51 - 1:05	Pass

	Spec	Test steps	Expected result	Actual result	Time	Pass?
4	2b	While running the program, monitor the RAM usage of the client	It should not exceed 50MB used for a prolonged period of time	It never exceeded 40MB used. At the point where the client runs for the longest period of time (timestamped), the device's memory use reaches a maximum of 6.93GB from a base of 6.89GB. The difference is 40MB - though even this usage was for a very short time period.	2:36 - 2:43	Pass
5	2c	While running the program, monitor the CPU usage of the client	It should not exceed 5% of CPU time usage for a prolonged period of time	While running the client for the longest period of time (timestamped), the <i>overall</i> CPU usage of the client did not exceed 1%!	2:36 - 2:43	Pass
6	3a	Send a file from the client to the server using only command line arguments	The file should be available on the server	The file is available on the server under the ServerReceived directory	2:36 - 2:59	Pass
7	3a	Request a file from the server using only command line arguments	The file should be accessible on the client now	The file is accessible on the client	3:44 - 4:10	Pass
8	3c	Run the programs	At no point should a graphical interface appear	No graphical interfaces appear	0:00 - 6:17	Pass
9	4b	While running the program, monitor the RAM usage of the server	It should not exceed 200MB used for a prolonged period of time	It never exceeded 2% of memory used. Since the laptop has 4GB of RAM, this works out to never exceeding 80MB of RAM	0:00 - 6:17	Pass

	Spec	Test steps	Expected result	Actual result	Time	Pass?
10	4c	While running the program, monitor the CPU usage of the server	It should not exceed 10% of CPU time usage for a prolonged period of time	It did not exceed 2% of CPU time for the most part. Between 1:13 - 1:15, it transiently spiked to 46% usage, but this did not last for more than 2 seconds.	0:00 - 6:17	Pass
11	6a	Send 5 files from the client to the server: two regular plaintext files, a plaintext source code file, a png image file, and an mp4 video file	The three plaintext files should have their file size significantly reduced, and the other files should not have their file size increased	The first plaintext files had its size reduced from 60kB to 32kB, the second from 160kB to 88kB, and the C# source code file from 16kB to 8kB. The other files did not have their size changed (2.3MB and 1.5MB respectively before and after transmission).	1:04 - 2:10, 2:33 - 3:40	Pass
12	6b	Retrieve those 5 files from the server	The files should be identical to before they were sent	The files were identical to before they were sent	3:43 - 4:50	Pass
13	8abc	Enter an invalid command line flag	The program should not crash, but rather display details on correct usage	The program did not crash and instead displayed details on correct usage	4:54 - 4:59	Pass
14	8abc	Enter in invalid command line arguments to the client program (after a client flag)	The program should not crash, but request interactive input from the user	The program did not crash and instead requested interactive input from the user	4:59 - 5:06	Pass
15	8a	Enter an invalid IP address interactively	The program should request a new IP address from the client, rather than attempting to proceed	The program requested a new IP address from the client	5:06 - 5:17	Pass

	Spec	Test steps	Expected result	Actual result	Time	Pass?
16	8a/c	Attempt to run the program with the server (-s) flag without specifying a valid port	The program should inform the user that they need to specify a port, and terminate	The program informed the user that they need to specify a valid (integer) port before terminating	5:18 - 5:28	Pass
17	8b/c	Attempt to run the client program, specifying an IP address where there is no server running	The program should interactively request a new IP address from the user	The program interactively requested a new IP address from the user	5:37 - 5:58	Pass
18	8b/c	Enter in a file path that does not exist	The program should inform the user of this and terminate without crashing	The program informed the user of this and terminated without crashing; the server also displayed a message confirming that the transmission was unsuccessful	6:03 - 6:17	Pass

The reasons for the tests above, and the conclusions that can be drawn from them, are listed in this table below.

Test	Purpose	Conclusions Drawn
1	To ensure that the client and server programs can interact with each other, and to show that the client program can work interactively (i.e. without all information entered through command line flags)	The client and server programs can connect to each other, the client interactive mode works
2	To ensure that the client can send a file to the server, and that the server can receive and store it, and to test again that the interactive mode on the client works	All is successful, the client can interactively send a file to the server, and the server can receive and store it properly
3	To ensure that the program can be run as one executable binary (rather than having to install dotnet and run the project with <code>dotnet run</code>)	It can be run through one executable
4	To ensure that the client program does not use too much RAM	The RAM usage of the client is within the boundaries set

Test	Purpose	Conclusions Drawn
5	To ensure that the client program does not use too much CPU time	The CPU usage of the client program on my client's laptop was as expected/within the bounds set throughout
6-7	To ensure that the program can be used without any interactive input, as requested by my client	The program works as expected with interactive input
8	To ensure there is no graphical interface for any functions, as requested by my client	There is never any GUI, as expected
9	To ensure that the server program does not use too much RAM	The RAM usage of the server is within the boundaries set
10	To ensure that the server program does not use too much CPU time	The CPU usage of the server program on my client's designated server laptop was as expected/within the bounds set throughout
11	To ensure that plaintext files have their sizes significantly reduced by compression, and to verify that other files do not have their sizes increased	The compression works as expected, being very effective in compressing plaintext files, and not increasing the sizes of the other files
12	Ensures that files can be retrieved from the server back to the client, and checks that the decompression algorithm works, since the files stored on the server are compressed but should be returned to the client uncompressed	The server can return files to the client correctly, and the decompression works as expected
13-18	Ensures that the program handles samples of invalid input robustly, without crashing	Unexpected and invalid inputs are handled gracefully as expected

4.3 Verifying the RSA Key Exchange

The two sections of the RSA Key Exchange procedure that I have sampled for formal testing are the Miller-Rabin Primality Test - which I use to generate numbers that are prime (to a very high probability) - and the Extended Euclidean Algorithm, which I use to generate the decryption constant, d , which is used to decrypt transmissions. In both cases, I compare the output from my algorithms with the widely trusted Bouncy Castle C# library [8], since these computations are too long to perform by hand (for checking).

4.3.1 The Miller-Rabin Primality Test

The following article [9] shows how Bouncy Castle can be used to test primality. Based on the information given there, I devised the following code (Listing 33) to test my prime number generation code (Listing 34). For this testing, the GenerateRSAPrime procedure has been made public.

Do note that Bouncy Castle also uses a probabilistic method to verify primes, similar to Miller Rabin. It is not feasible to deterministically verify the primality of these numbers, since the primes generated by my algorithm are 512 bits in length, corresponding to a 155 digit decimal number!

```

1  int fails = 0;
2
3  for (int i = 0; i < 500; i++)
4  {
5      Org.BouncyCastle.Math.BigInteger p = new
        Org.BouncyCastle.Math.BigInteger(1,
        RSA.GenerateRSAPrime(65537).ToByteArray(true, true));
6
7      if (p.IsProbablePrime(100))
8      {
9          Console.WriteLine("Pass: " + p + " is prime");
10     }
11     else
12     {
13         Console.WriteLine("FAIL: " + p + " is NOT prime");
14         fails++;
15     }
16 }
17
18 Console.WriteLine("Failed " + fails + " times.");

```

Listing 33: Code for testing prime number generation

```

1  using System.Security.Cryptography;
2  using System.Numerics;
3
4  static class RSA
5  {
6      public static BigInteger GenerateRSAPrime(BigInteger e) // e must
        be prime in this implementation!
7      {
8          while (true)
9          {
10             BigInteger bigInt;
11
12             do bigInt = GenerateOdd512BitInt();
13             while (bigInt % e == 1); // ensures e will be relatively
        prime with phi(n), as e used is prime
14
15             if (MillerRabin(bigInt, 40)) return bigInt;
16         }
17     }
18
19     private static BigInteger GenerateOdd512BitInt()
20     {
21         byte[] randomInt = new byte[64]; // 512 bits = 64 bytes
22         RandomNumberGenerator.Fill(randomInt);

```

```

23         randomInt[0] |= (byte)(128); // sets MSB to 1, so num is 512
24             bits
25
26         randomInt[63] |= (byte)(1); // sets LSB to 1 - odd
27
28         return new BigInteger(randomInt, true, true); // isUnsigned,
29             isBigEndian
30     }
31
32     private static BigInteger GenerateInRange(BigInteger lower,
33         BigInteger upper)
34     {
35         BigInteger num = -1;
36
37         while (num < lower || num > upper)
38         {
39             byte[] randomInt = new byte[64]; // 512 bits = 64 bytes
40             RandomNumberGenerator.Fill(randomInt);
41
42             num = new BigInteger(randomInt, true, true);
43         }
44
45         return num;
46     }
47
48     private static bool MillerRabin(BigInteger n, int rounds)
49     {
50         int s = 1;
51         while ((n-1) % (BigInteger)Math.Pow(2, s+1) == 0) s++;
52
53         BigInteger d = (n-1)/(BigInteger)(Math.Pow(2,s));
54
55         for (int i = 0; i < rounds; i++)
56         {
57             BigInteger a = GenerateInRange(2, n-3);
58
59             BigInteger x = BigInteger.ModPow(a, d, n);
60
61             for (int j = 0; j < s; j++)
62             {
63                 BigInteger y = BigInteger.ModPow(x, 2, n);
64
65                 if (y == 1 && x != 1 && x != n-1) return false;
66
67                 x = y;
68             }
69
70             if (x != 1) return false;
71         }
72     }

```

```

71     return true;
72 }
73 }

```

Listing 34: Prime number generation code

The test was successful, with all 500 test cases passing. Listing 35 provides a sample of the output. The full output file will be attached to the final submission.

```

1 Pass:
2 1022838897525948452326314761450558643029776665031161991762900964759
3 9208738424303376369362395777467244295217649583342769169135583944463
4 959920972181708117653 is prime
5 Pass:
6 7055327374140629097848086175954549538219546373088930332518480276112
7 9612896958072263110059929438392097574740738361435914807912613842778
8 89017074947936742903 is prime
9 Pass:
10 8207953070121178983353661236359241115325034429146888551795042317834
11 7923721765858790424803834171997353892302245591864022636164346689587
12 71189992592692610913 is prime
13 Pass:
14 1021332556497042893944292189847070889341644895565219608225057160450
15 3999386667064122613896042792383724465008354104534896671022129077163
16 694172032571219992307 is prime
17 Pass:
18 1274402175016710989923704888496482876377589152883867932068249982390
19 8622666336558289318584767846048723132582186417734332379160181912285
20 144992739219631937129 is prime
21 Pass:
22 9764341766759844351848046604572785689377905992592986091796999794005
23 8297117965675009692792015651758074253737380136026572180414929411612
24 71880072376029007473 is prime
25 Pass:
26 1029199440267379033809149413870576337928811811179278355597401005662
27 2969700609653395112222932344007990988742724898641565867000959568425
28 023704799305892705049 is prime
29 ...
30 Failed 0 times.

```

Listing 35: Prime number generation test output

4.3.2 The Extended Euclidean Algorithm

I similarly tested my implementation of the Extended Euclidean Algorithm by confirming that my decryption constant, d , generated satisfies the equation $de = 1 \pmod{\phi(n)}$. Again, I used the Bouncy Castle library for this.

Do note that the BigInteger data types I used in my code - the default System.Numerics.BigInteger - differs from the Org.BouncyCastle.Math.BigInteger, and some conversion needs to take place

between these data types for the testing.

Listing 36 shows the code for the testing, and Listing 37 shows the code being tested. Again note that the `GetModMultInverse` method has been made public for the purposes of this testing, and relevant parts of the `GenerateKeys` method have been copied into the testing code.

```

1  int fails = 0;
2
3  for (int i = 0; i < 500; i++)
4  {
5      System.Numerics.BigInteger e = 65537;
6
7      System.Numerics.BigInteger p = RSA.GenerateRSAPrime(e);
8      System.Numerics.BigInteger q = RSA.GenerateRSAPrime(e);
9      System.Numerics.BigInteger n = p*q;
10
11     System.Numerics.BigInteger phin = (p-1)*(q-1);
12
13     System.Numerics.BigInteger d = RSA.GetModMultInverse(e, phin);
14
15
16     Org.BouncyCastle.Math.BigInteger bouncyCastlePhin = new
17         Org.BouncyCastle.Math.BigInteger(1, phin.ToByteArray(true,
18             true));
19
20     Org.BouncyCastle.Math.BigInteger bouncyCastleE = new
21         Org.BouncyCastle.Math.BigInteger(1, e.ToByteArray(true,
22             true));
23
24     Org.BouncyCastle.Math.BigInteger bouncyCastleD = new
25         Org.BouncyCastle.Math.BigInteger(1, d.ToByteArray(true,
26             true));
27
28     if
29         (bouncyCastleE.ModInverse(bouncyCastlePhin).Equals(bouncyCastleD))
30     {
31         Console.WriteLine($"Pass: ({n}, {e}, {d}) is a valid key");
32     }
33     else
34     {
35         Console.WriteLine($"FAIL: ({n}, {e}, {d}) is NOT a valid key");
36         fails++;
37     }
38 }
39
40 Console.WriteLine("Failed " + fails + " times.");

```

Listing 36: Code for testing my Extended Euclidean Algorithm

```

1  public static Tuple<BigInteger, BigInteger, BigInteger> GenerateKeys()
2  {
3      BigInteger e = 65537;
4
5      BigInteger p = RSA.GenerateRSAPrime(e);

```

```

6     BigInteger q = RSA.GenerateRSAPrime(e);
7
8     BigInteger n = p*q;
9     BigInteger phin = (p-1)*(q-1);
10
11     BigInteger d = RSA.GetModMultInverse(e, phin);
12
13     return new Tuple<BigInteger, BigInteger, BigInteger>(n,e,d);
14 }
15
16 public static BigInteger GetModMultInverse(BigInteger a, BigInteger m)
17 // uses the extended euclidean algorithm to find x s.t. ax = 1 (mod
18 m)
19 {
20     BigInteger r_0 = a;
21     BigInteger s_0 = 1;
22     BigInteger t_0 = 0;
23
24     BigInteger r_1 = m;
25     BigInteger s_1 = 0;
26     BigInteger t_1 = 1;
27
28     BigInteger r_2, s_2, t_2;
29
30     do
31     {
32         BigInteger q = r_0 / r_1;
33
34         r_2 = r_0 - q * r_1;
35         s_2 = s_0 - q * s_1;
36         t_2 = t_0 - q * t_1;
37
38         (r_0, r_1) = (r_1, r_2);
39         (s_0, s_1) = (s_1, s_2);
40         (t_0, t_1) = (t_1, t_2);
41
42     } while (r_2 != 0);
43
44     BigInteger d = s_0 % m;
45     if (d < 0) d += m;
46
47     return d;
48 }

```

Listing 37: Extended Euclidean Algorithm code being tested

The test was successful, with all 500 test cases passing. Listing 38 provides a sample of the output. The full output file will be attached to the final submission.

```

1 Pass :
2 (149349778010361168172503824807965381656766847255558258080525297566
3 7355400724953289104848232464010197346062245815840744230862356924158

```

```

4 0889000216169436892525907304695951881476130028382493615107183356805
5 4347068252327600168642507154327679699155953826221009109910333876034
6 328701167950110619672925451790368891547069, 65537, 1466629997282576
7 8743222914013444368879665228429243356231665238110990686006356187158
8 4310881707918837116129627563385383244899582780604797725601707773108
9 2486266490816520079476450782221857942293291206624691700166578537916
10 8528521088931345221499071517542771704314220824661639625073697177421
11 6026958532617412304157993) is a valid key
12 Pass:
13 (807418634561660175134634061191577918018531741281873331117543254971
14 2154299890095844551892301953037307760256381821788104174100813549835
15 4208257553190770856166317149710702681106081950435319605772128692621
16 5305211836827630088955466137148133197816977939514076758730247261367
17 42400596345711420526694182733579094499727, 65537, 24874166092131038
18 8573908810221065324393764680355845134126725335579425965965456208098
19 4828502623431393620086002547902739265988608352093888131467718878898
20 3103210226713408543891189561728425036799813806098642979175577198858
21 8892448372674993741612294687157692988546949275905178163138617458777
22 9269090661930495240585) is a valid key
23 Pass:
24 (640134708856914573688808137630790462891404616025642597893919016249
25 8266796233138575220133172813986095103726698940018565943441946363543
26 6819336434020550414216361865827132152135344802543642591729906588854
27 5963579063858961537706131532850221329251803630420635785663696771412
28 40548745636031636479345938992375593328217, 65537, 41975966420687403
29 7631819120720863331290082142510368046210402821663126196756212719944
30 5888934520668514535036008467694552259325535880087427424177566921210
31 9957101295599619100671622730273923425031259260535275551457701342928
32 1178680888493179333737322642703882215837939063360080267362341272356
33 80109693217092808509773) is a valid key
34 Pass:
35 (130311951598420955714137024806731651809460618800365141486522006331
36 4901894354818698720788187032377314330983923828270957517012008940084
37 1604143989608873449755714832347522365891298048974831127793014335963
38 1639969739145035936198050626839110498139125784089032411193550057002
39 054325086734831979789986166262990701277313, 65537, 1174272994727855
40 4681492577130492929430567947517422469995223354941359561953539607838
41 3742905475641404172368154766812942212921217345887895786491843436721
42 1379966153698216981041295476211195845649213831344686277362841373883
43 6134824569774092956558624616077099271643792577765094964372385492502
44 5785756907382573834839061) is a valid key
45 ...
46 Failed 0 times.

```

Listing 38: Extended Euclidean Algorithm test output

5 Evaluation

5.1 Evaluation Against the Requirements

The original specification, from the Analysis, is copied again below:

1. The program should allow the user to create file backups on a server, run by the client themselves,
 - (a) The software should be able to run on both the client and server, with appropriate functionality for each situation,
 - (b) They should be able to work together, with a server being able to store the user's backed up files,
2. The client program should run on modern Linux systems, usable on a laptop or desktop computer,
 - (a) An installable binary package should be created that is executable on amd64 Linux systems,
 - (b) It should be very unlikely for the client to use more than 50MB of RAM for prolonged periods of time,
 - (c) It should be very unlikely for the client to use more than 5% of CPU time for prolonged periods of time while running on a modern laptop; my client's processor, the Intel i9-12950HX, can be used for a test case,
3. It should be usable with an intuitive command line interface, not requiring interactive input,
 - (a) All required functions of the program should be able to be passed in through command line flags and parameters,
 - (b) There can be an optional interactive mode, but this should not allow any functionality than the non-interactive mode does not,
 - (c) There should be no graphical interface,
4. The server should be able to run on a low-end laptop 10-15 years old,
 - (a) The program should avoid reading and writing to the system drives as much as possible, since most such devices use HDDs,
 - (b) It should be very unlikely for the server program to use more than 200MB of RAM for prolonged periods of time,
 - (c) It should be very unlikely for the server program to use more than 10% of CPU time for prolonged periods of time while running on such a device; my client's desired server's processor, a 2nd generation Intel i5 processor, can be used as a test case,
5. All file transfers should be secure, so a man-in-the-middle would not be able to intercept data,

- (a) Any data intercepted should be of a format that the interceptor would not be able to decipher, or gain any useful information from,
 - (b) The methods used should have their correctness checked,
6. Text files stored on the server should be compressed such that they use a reasonable amount of storage,
- (a) They should be at most as large as (or very close to) the uncompressed files, i.e. the file size should never increase following compression by more than 10 bytes,
 - (b) The client should be able to decompress any file when needed by the user, storing any needed details about the compression within the file,
7. File transfers should be verifiably correct, so data is not corrupted during transmission,
- (a) Once a file has been transferred to the server, it should be compared with the original file on the client device,
 - (b) This checking should be with a much smaller version of the file than that which was originally sent for most files – it should be up to 10kB in size,
8. Generally, the program should work robustly,
- (a) Problems with user input or command line arguments should be gracefully handled,
 - (b) The program should not crash while running,
 - (c) Errors should be reported to the client, server, or both, depending on which is appropriate in the situation.

This table, below, details the extent to which the specification points have been achieved.

Spec	To what extent has it been achieved?
1a	This has been fully achieved. The same program can be run on the client and server, specifying which it should act as through a flag. The client will make a connection to the server running at the IP address specified by the client.
1b	This has been fully achieved. The client can send files to the server, and the server stores them under the <code>ServerReceived</code> directory, maintaining the file structure of the client. For example, if the client sends the file <code>/home/shayan/test1.txt</code> , it will be stored at <code>ServerReceived/home/shayan/test1.txt</code> on the server.
2a	This has been fully achieved. The program comes with a self contained executable binary for amd64 Linux systems.
2b	This has been fully achieved. In all of my testing the client program never used more than 40MB of RAM, below the limit of 50MB set. This is partly due to my choice of algorithms, for example, opting to use the simpler Huffman Coding rather than Prediction by Partial Matching, which would use more RAM.

Spec	To what extent has it been achieved?
2c	This has been fully achieved. In my testing the client program only exceeded the 5% limit very briefly, which is within the bounds of the specification point. For the vast majority of its runtime, the program used less CPU time than that. Do note that this specification point is limited to my client's laptop and its processor - CPU usage will vary greatly by device.
3a	This has been fully achieved. The server program (running the program with the server -s flag) accepts no interactive input and requires the port to be passed in as a command line argument. The client program (running the program with the client -cr or -cs flag) has all of its functionality accessible through command line arguments.
3b	This has been fully achieved. When running the program with the client -cs or -cr flag, the server IP, server port, and file paths can be specified interactively. This allows the user to correct errors identified by the program. However, no core functionality is gained through the interactive mode so the program can be used in scripts without any problems.
3c	This has been fully achieved. There is no graphical interface and the program only works through a console, as intended.
4a	This has been fully achieved. Files are compressed before being sent, and are stored in compressed form on the server, only decompressed on the client device after being retrieved. This minimises how much data the program reads and writes from the server's hard drive.
4b	This has been fully achieved. In all of my testing the client program never used more than 80MB of RAM, below the limit of 200MB set.
4c	This has been fully achieved. In my testing the client program never exceeded the 10% limit. In fact, it never exceeded 2% processor usage. Do note that this specification point is limited to my client's server laptop and its processor.
5a	This has been fully achieved. First, an asymmetric RSA key exchange is used to create public and private keys to share a symmetric AES key between the client and server devices. From that point on, all data is encrypted symmetrically with that AES key.
5b	This has been fully achieved. A standard C# implementation of AES is used, which is widely recognised as correct, and I have checked key components of my RSA implementation through testing.
6a	This has been fully achieved. Plaintext files are almost always significantly compressed, and all other files are kept the same size. There is also functionality to ensure that text files never increase in size unnecessarily.
6b	This has been fully achieved. Files are decompressed by the client once they are received. This has been shown in testing, where the files are identical after receipt to before being backed up.
7a	This has been fully achieved. A hash of the file is sent along with the file itself, and the recipient verifies integrity by comparing hashes. This is implemented for both sending and retrieving files.
7b	This has been fully achieved. The hash is 256 bits (32 bytes) in size, which is much smaller than the 10kB limit.

Spec	To what extent has it been achieved?
8a	This has been fully achieved. If any user input is invalid, the user is informed of this, and in some cases is given the opportunity to rectify it.
8b	This has been fully achieved. In all of my testing, the program has never crashed. Errors are handled gracefully through exception handling and defensive programming.
8c	This has been fully achieved. The program never fails silently; there is always some form of error message displayed.

As justified in the table, all of my specification points have been fully achieved.

5.2 Client Feedback

To gather some feedback from my client, I conducted a final interview with them, to get their opinion on the final product made.

5.2.1 Final Client Interview

Q: Overall, what do you think of the backup utility that I've created?

A: I think it's broadly excellent. It does what I want it to do - I can use my old laptop as a server to store backups of files on my main laptop. I love that all the file transfers are encrypted, and that the files are sent compressed. It has a very intuitive command line interface, and so ties in really neatly with my workflows, which are normally based in the terminal. Having the single executable binary means that I can very easily install it system-wide, and I am really pleased that all of its functions are available through command line environments, since I want to incorporate this into a few of my system administration scripts.

Q: Thank you so much! Are there any areas where you see shortfalls?

A: There are some small improvements that I would suggest. Do note that these are mostly matters of personal preference. Firstly, when the server is running, if it is listening on the broader private network (i.e. not on the loopback interface) it reports its IP address as 0.0.0.0. While this is not incorrect, as it signifies that the server is accessible on both its private IP address on the private network, and from the wider internet, since I will only use the server from a private network, I would rather that it displays the private IP address of the laptop on the network. Currently, if I forget the server's private IP address, I would need to run separate commands to determine that. Another point is that there is no indication on the server when a file has been successfully received. Though I can see this on the client, since the program terminates once all of the files have been sent, it would be nice to have some indication of this on the server too, for my own sanity. Finally, while I really like that there are no graphics, currently command line arguments are just specified sequentially. I think it could be most intuitive if they followed flags. For example, instead of just needing to enter the server IP address and port after the -cs [client

send] flag, perhaps something like this: `-cs --serverip [server IP] --serverport [server port]` could be more intuitive. [The client wrote the italicised text on paper during the interview.]

Q: Great, that's really good to know! Besides from opinions of preference, are there any features you think should have been implemented?

A: Well, you implemented everything I told you to, and I am very happy with the outcome, particularly given the deadline I set you, which you have even exceeded!

Q: Thanks! But if you were to request the software again, and you no longer needed to worry about your deadline, are there any additional features you would have asked for?

A: Yes, there are actually a few points. I think it could have been helpful to have an option to back up all the files in a given directory at once. For example, I would like to be able to specify `/home/sam/` and have the program backup my whole home directory. This isn't a big deal though, as since it can be used as part of a script, I can easily create a script to do this for me. Another useful feature could be the ability to store multiple versions of a file on the server, so I can see the different versions of my files over time. Following on from this, when backing up a file, if it hasn't changed since the last backup, there could be some functionality to just keep the old copy, reducing runtime, network load, and the strain on my old hard drives. This is also useful as a standalone point, even if old versions can't be kept - if a file hasn't changed since the last backup, it shouldn't be re-sent ideally. Finally, while I really like the encryption implemented to prevent malicious parties being able to view the data I am sending, it could also have been useful to have some method of verifying the authenticity of communications between the client and the server, to ensure that a malicious actor can't retrieve my backups!

Q: Thank you for that - that's really useful. To wrap up, could you rate the overall backup utility out of 10 based on its ease of use, functionality, and overall fit to your requirements.

A: Of course! I would give it a 9/10 for ease of use. I love the purely command line based interface, that it can run with just one executable binary, and the fact that all needed options can be passed in through the command line. The only point of improvement here is that I would have preferred the command line options to be passed in through flags, as I pointed out before. For functionality, I would give it an 8/10. It contains absolutely everything I needed, but there is of course more that could be added, as I recently mentioned, in the grand scheme of things. I think it is a 10/10 for the fit to my requirements. It achieved every point I indicated at the start of the project, and all in the time period I had designated. I am very happy with the outcome overall!

5.2.2 Evaluating the Interview

Overall, I am very pleased that my client was impressed with the software produced. He seemed to really like the general functioning of the backup utility, and made many positive comments

about points that he had brought up in the initial client interview being met. I was also very pleased to hear that he considers the program to have fully met its specification.

However, there were also constructive points mentioned, about potential improvements that could be made. These fall under two broad categories. Firstly, there are improvements based on what they would prefer. These points were that they would like:

1. The server's private IP address displayed when it is listening on the private network,
2. The server to give a printed indication whenever a file is successfully received, and
3. The program to accept command line arguments with named flags, rather than just sequentially.

Secondly, there are improvements that include adding new functionality. These suggestions were:

1. To enable the utility to store backups of all the files under one directory natively, by just specifying the parent directory,
2. To allow the user to store multiple versions of a file as backups, rather than just the most recent copy,
3. To only send a file to the server if it has changed since the previous backup, rather than sending it to overwrite an identical file in some situations, and
4. To incorporate some sort of verification that the client or server is who they claim to be, so malicious actors cannot impersonate them.

The first group of improvements can be implemented really simply, by just making minor tweaks to the code, since the overall functionality is not changed, just the way it interfaces with the user. However, more substantial additions are needed for the functionality suggested in the second group. These will be explored in depth in the next section.

5.3 Improving the Outcome

As identified by my client, and described in the previous section, there are four points of functionality that could be added to improve the outcome if the problem was revisited.

Firstly, the backup utility should be able to store backups of all of the files under one directory natively. This should be fairly straightforward to implement, using the built-in C# `Directory.GetFiles()` method to get the names of all of the files under the directory specified by the user. At that point, each file should be treated as it would be had the client entered it separately. There would be a need to differentiate between when the user is specifying a directory, under which all files should be backed up, or just a file itself to back up. This distinction could be made by requiring directories to be entered with a leading `/`. For example, specifying `/home/shayan/` would back up all of the files under the `/home/shayan/` directory, while specifying `/home/shayan/test1.txt` would only back up the `test1.txt` file.

The second point mentioned was allowing the user to store multiple versions of a file as backups. This is considerably more difficult to implement, as it would involve changing the way files are stored on the server. Currently, files are stored according to their name and location on the client device. One option for implementing this could be as follows. Files could be stored as directories containing various versions of that file (by timestamp) on the server. This would mean that, for example, the file `/home/shayan/test1.txt` would be stored on the server at `ServerReceived/home/shayan/test1.txt/1774347692`, where `1774347692` is the UNIX timestamp when the backup is made. In this way, when retrieving a file, the server can access every stored version of a file, and send them to the user. Perhaps first the server can send a list of available timestamps to the client device (with the client translating it to a human-readable date format), and then the user can choose which backup timestamp they would like to have retrieved, sending that timestamp to the server, with the server then finally returning the file. Perhaps there could also be a command line flag to specify that the n^{th} most recent backup should be returned, to prevent a reliance on the interactive mode, and allow the program to continue being used in scripts.

This makes it more important to have a way of deleting backups from the server. There could be specific types of messages that the client can send to the server to indicate that it should delete a given backup (either the n^{th} backup, the oldest backup, or the backup with a certain timestamp for instance). Additionally, the server program could be configured with command line flags to only store the most recent n backups of any given file, with it deleting the oldest backup once a new backup has been received that brings the total number of backups for that file to $n + 1$. These additions would help manage the disk space is used on the server.

The third point mentioned by my client was that a file backup should only be sent from the client to the server if there is not already an identical file stored on the server. For example, if `ServerReceived/home/shayan/test1.txt` exists on the server, and the user requests to back up `/home/shayan/test1.txt`, and the contents of the two files are identical, then the file should not be needlessly sent to the server. However, there needs to be a way to check this. The already implemented hashing algorithm could be used for this. The client would send the name of the file to back up, as usual, but then the server could send a hash of the most recent backup of the file, if one exists (otherwise it could send a series of 256 zero bits for example, or some other form of message indicating the file is not present). Then, the client would compute a hash of the file it wants to back up, and compare it to the hash it received from the server, and only send the actual file to the server if the hashes are not identical. If they are, it could send some form of message indicating that, so the server does not expect a file. This could take the form of a single bit sent from the client to the server. For instance a 0 could indicate that the files are identical and so the server should expect nothing more, and a 1 could indicate that the file to store will follow.

The final point mentioned was verifying that the devices are who they claim to be during transmissions. For this, I would need to implement digital signatures and signed certificates. This would require me to slightly change the way my program is structured. Currently, whenever a client device connects to the server, the server generates a new pair of RSA keys, using that key pair to generate the AES key that will be used for the remainder of those communications. In order to be able to verify authenticity, I would need to use the same RSA algorithm to generate

a new key pair. The private key will be kept secret, but the public key will be put inside the program code. I would then need to generate certificates for any client devices, and the server, that my client wishes to use, signing these with that private key that has been generated. The certificates can be as simple as containing the public key of the device the owns the certificate, and a hash of they key that is encrypted using the aforementioned private key. Then, upon each connection, the two devices would send each other their signed certificate. By decrypting the hash in the certificate with the known and trusted public key that has been put in the program source code, the receiver can compare this with the sender's public key in the certificate. If they match, then the receiver can be assured that the sender is genuine, since they will need to have had access to trusted private key at some point. One of the public keys can then be used to establish the shared AES key, with the rest of the communications carrying on as normal. Do note that for this to work, the server can no longer generate a new RSA key pair for each connection; instead, each device in the system will need to have a fixed, pre-generated key pair, so that their certificate can contain their public key. Otherwise, they would need a new certificate signed before each transaction!

References

- [1] Billy B. Brumley et al. “Practical realisation and elimination of an ECC-Related software bug attack”. In: *Proceedings of the 12th Conference on Topics in Cryptology. CT-RSA’12*. San Francisco, CA: Springer-Verlag, 2012, pp. 171–186. ISBN: 9783642279539. DOI: [10.1007/978-3-642-27954-6_11](https://doi.org/10.1007/978-3-642-27954-6_11). URL: https://doi.org/10.1007/978-3-642-27954-6_11.
- [2] kartik. *RSA Algorithm in Cryptography*. 23rd July 2025. URL: <https://www.geeksforgeeks.org/computer-networks/rsa-algorithm-cryptography/> (visited on 26/10/2025).
- [3] Wikipedia contributors. *Extended Euclidean algorithm* — *Wikipedia, The Free Encyclopedia*. [Online; accessed 16-February-2026]. 2026. URL: https://en.wikipedia.org/w/index.php?title=Extended_Euclidean_algorithm&oldid=1335653676.
- [4] Andrew D. Ker. *Computer Security Lecture Notes*. <https://www.cs.ox.ac.uk/andrew.ker/docs/computersecurity-lecture-notes-mt2014.pdf>. 2014.
- [5] Wikipedia contributors. *AKS primality test* — *Wikipedia, The Free Encyclopedia*. [Online; accessed 23-January-2026]. 2025. URL: https://en.wikipedia.org/w/index.php?title=AKS_primality_test&oldid=1328685557.
- [6] Wikipedia contributors. *Miller–Rabin primality test* — *Wikipedia, The Free Encyclopedia*. [Online; accessed 16-February-2026]. 2025. URL: https://en.wikipedia.org/w/index.php?title=Miller%E2%80%93Rabin_primality_test&oldid=1320502394.
- [7] Professor Vijay Raghunathan. *ECE264: Huffman Coding*. 2017. URL: <https://engineering.purdue.edu/ece264/17au/hw/HW13?alt=huffman> (visited on 10/11/2025).
- [8] bcgit. *bc-csharp*. 9th Jan. 2026. URL: <https://github.com/bcgit/bc-csharp> (visited on 22/03/2026).
- [9] William J Buchanan. *Test for Prime Number with C# and Bouncy Castle*. Accessed: March 22, 2026. Asecuritysite.com, 2026. URL: https://asecuritysite.com/primes/testprime_csharp.